# changelog

24 Sep 2026: correct “min-max” to “max-min”

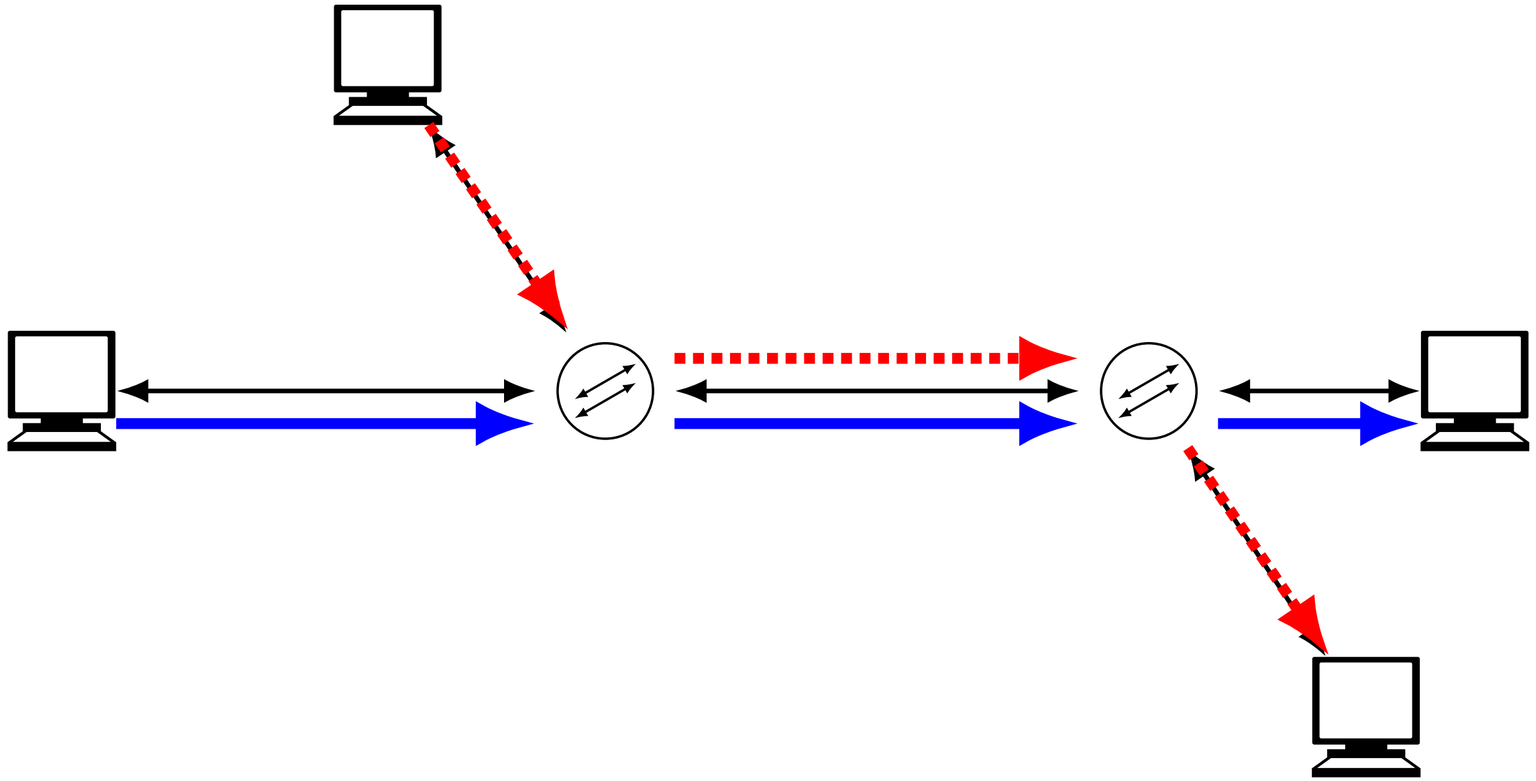
# congestion: sharing

want to consider multiple flows

key questions:

- is it stable if both flows changing window sizes?
- is there one winner/loser?
- is the winner/loser who we want it to be?

# exericse: what should happen?



two connections on shared link

intuition: each should get half of link bandwidth

question: will this happen if they don’t start equal?

# showing AIMD

slides based on Chiu and Jain, “Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks”’

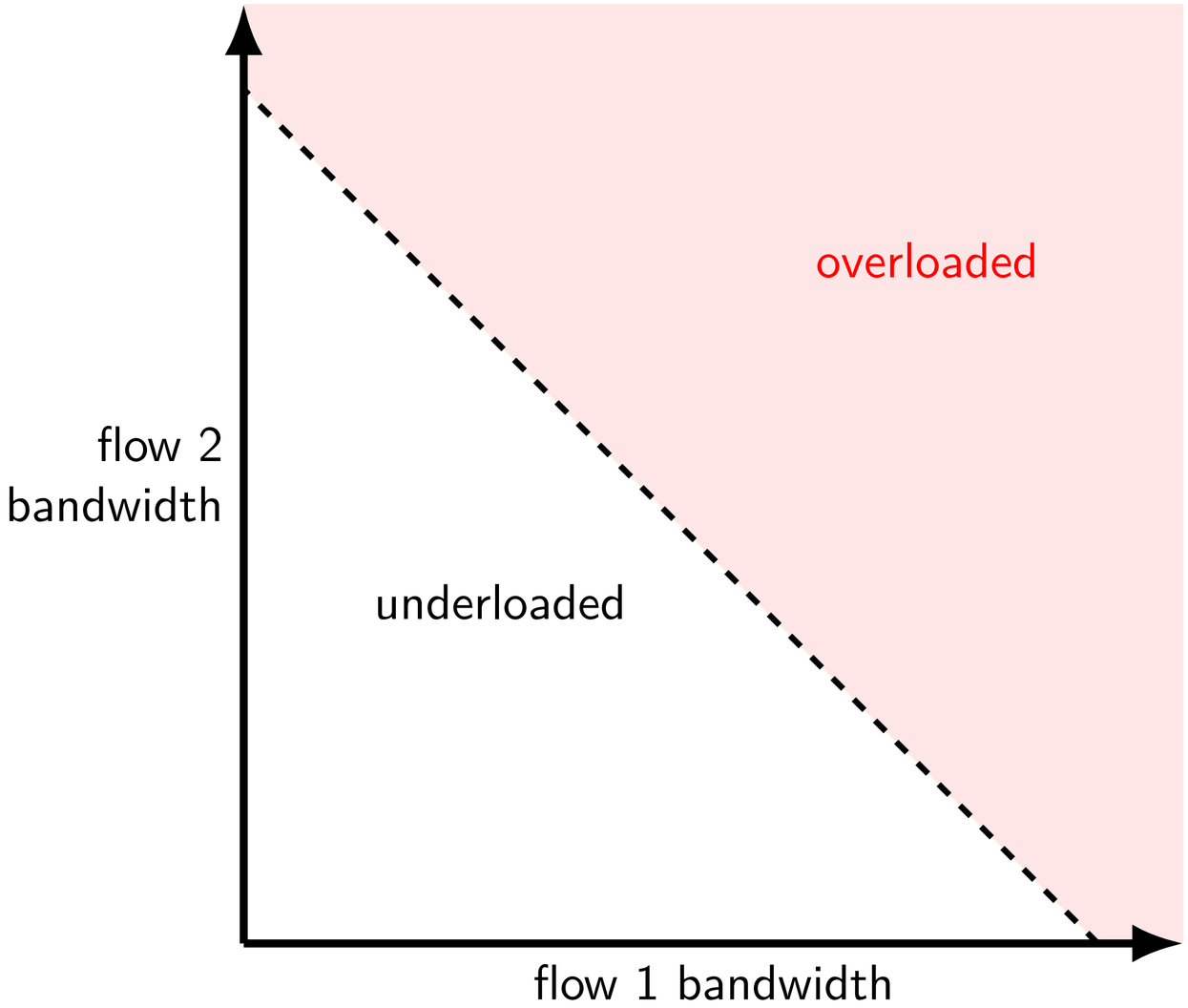
1989 paper

you might notice 1989 is well after TCP was in use

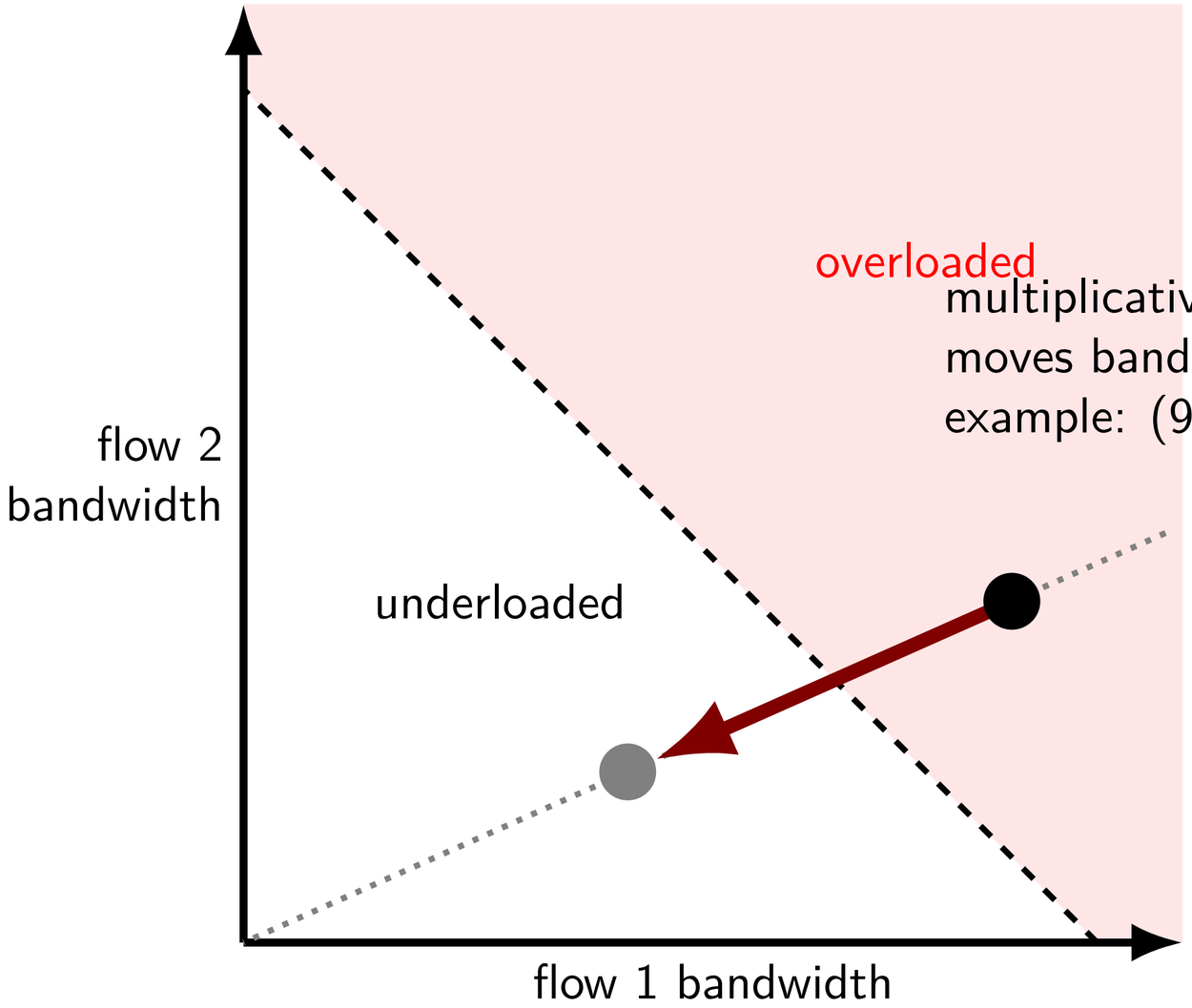
(kinda deployed without all the theory being developed…

… and it’s still not really a solved problem)

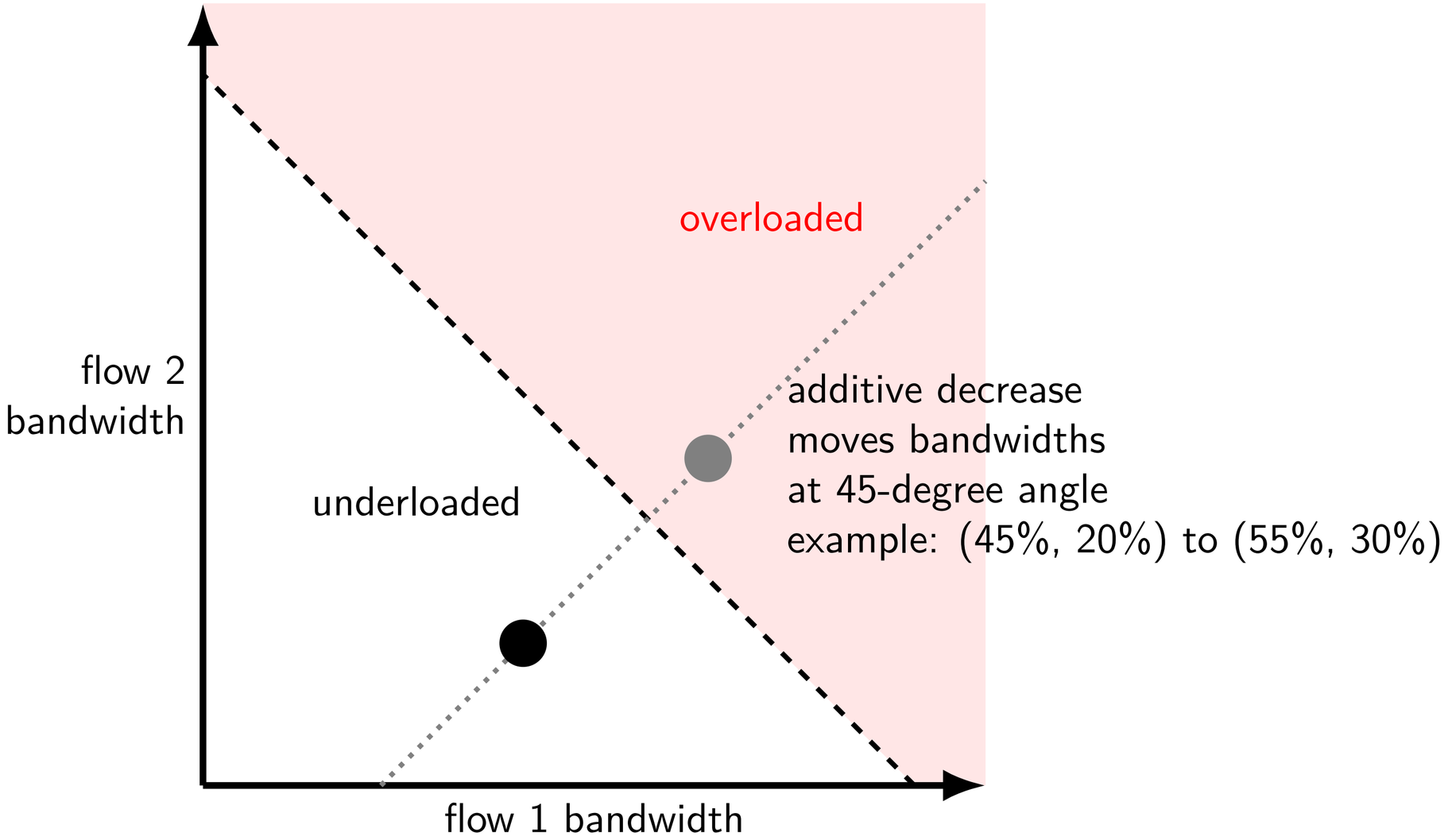
# picturing sharing



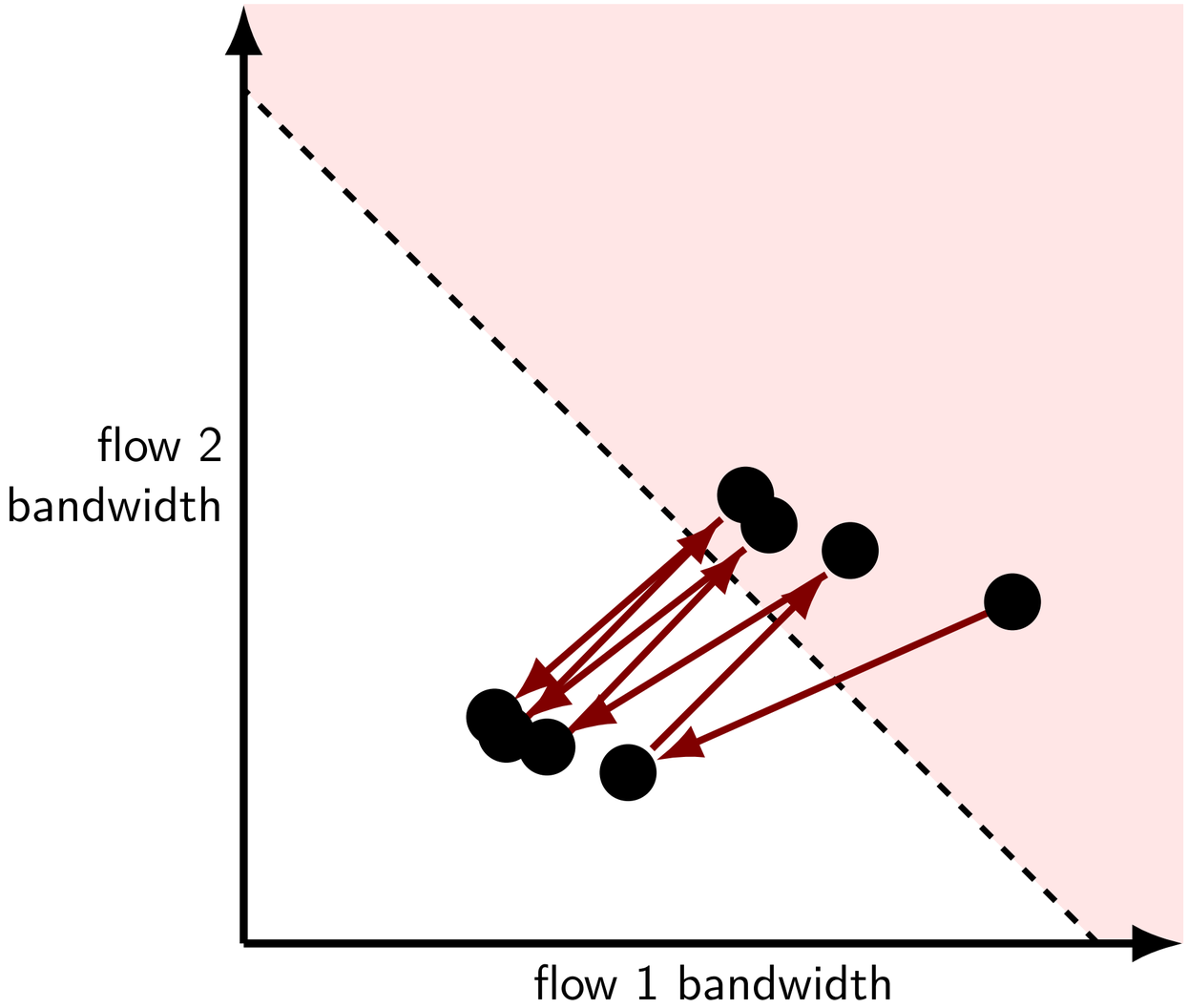
# picturing sharing



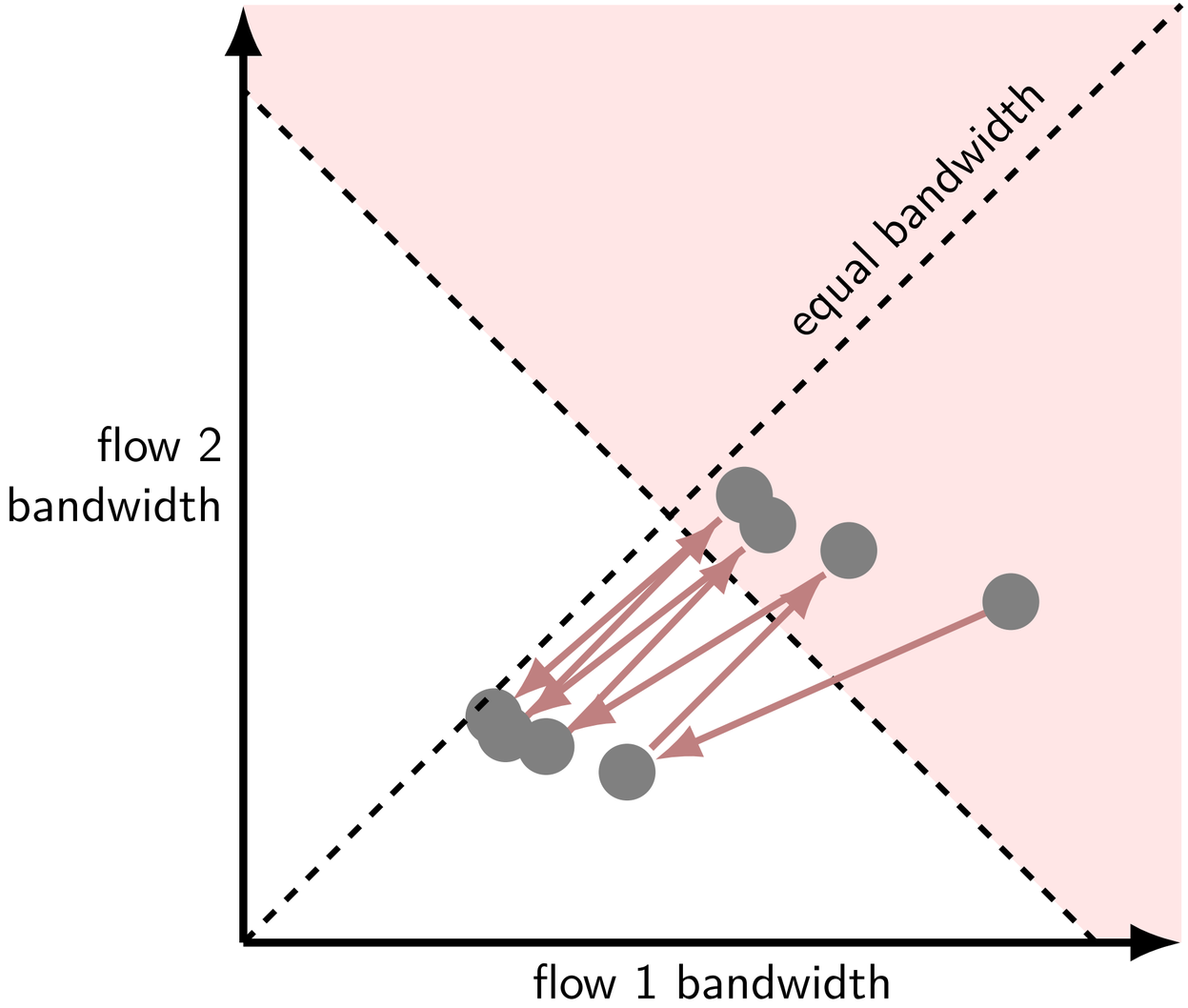
# picturing sharing



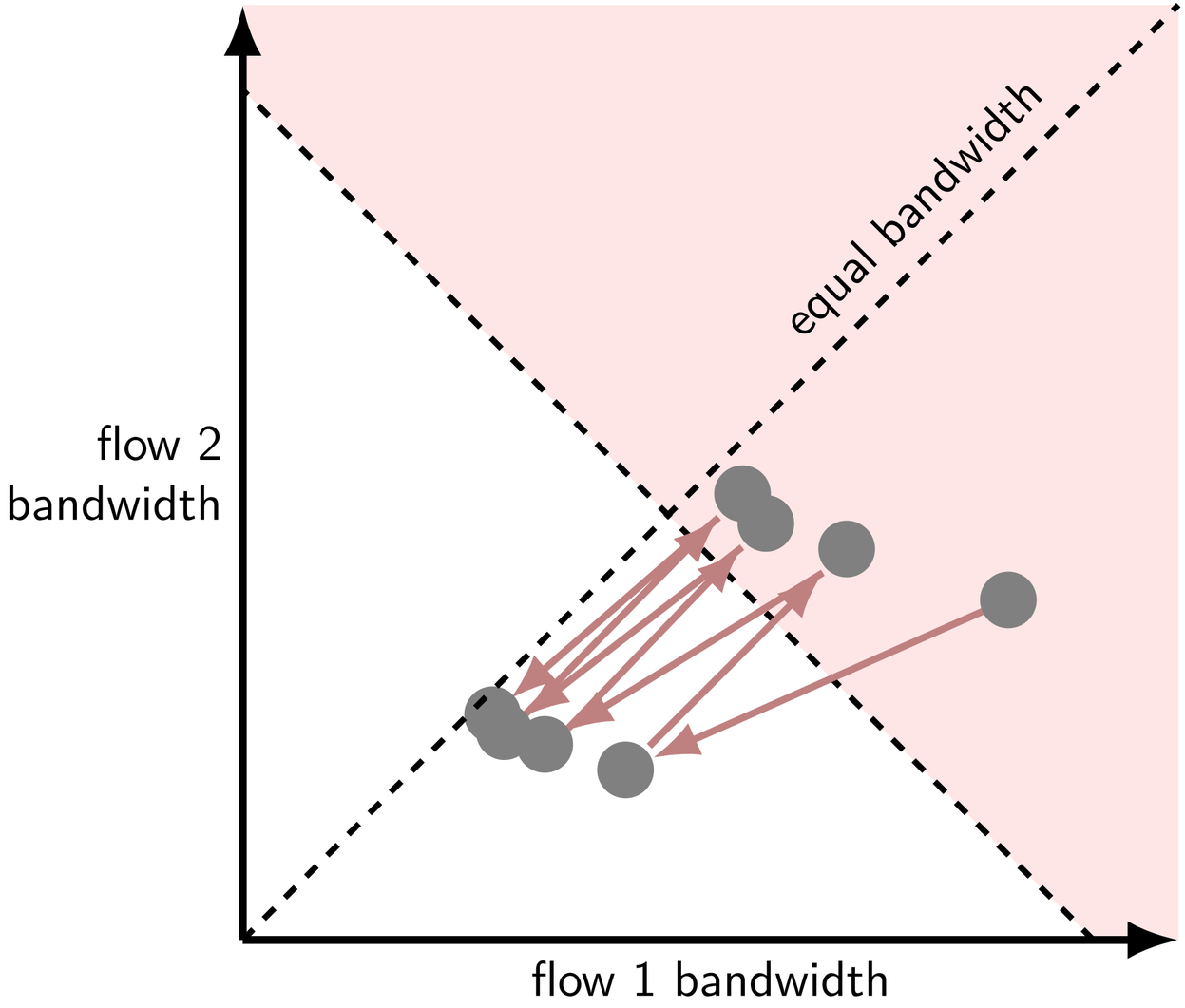
# picturing sharing



# picturing sharing



# picturing sharing




multiplicative decrease brings closer to equal bandwidth line

additive increase keeps same distance from line

# some assumptions we made

… that are probably not true

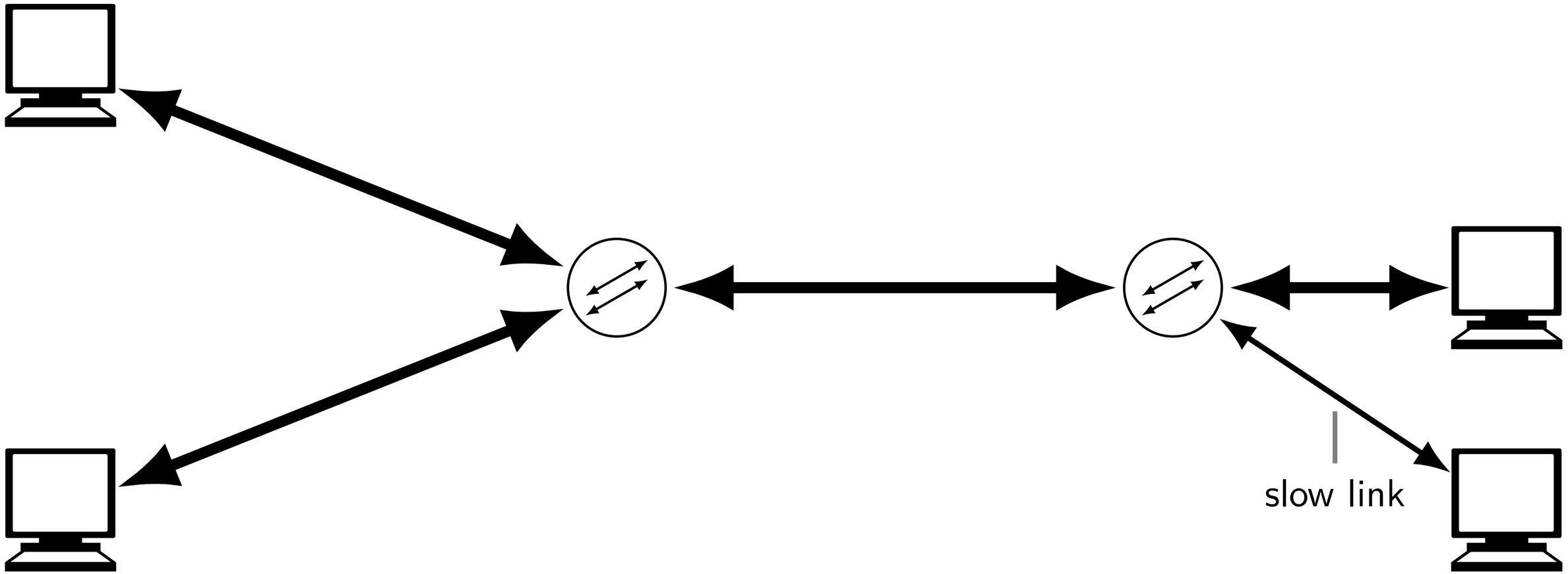
both flows experience drops equally when network overloaded

- higher-bandwidth flow more likely to see drops?
- depends on how switches manage queues?
- depends on how ‘bursty’ hosts are?

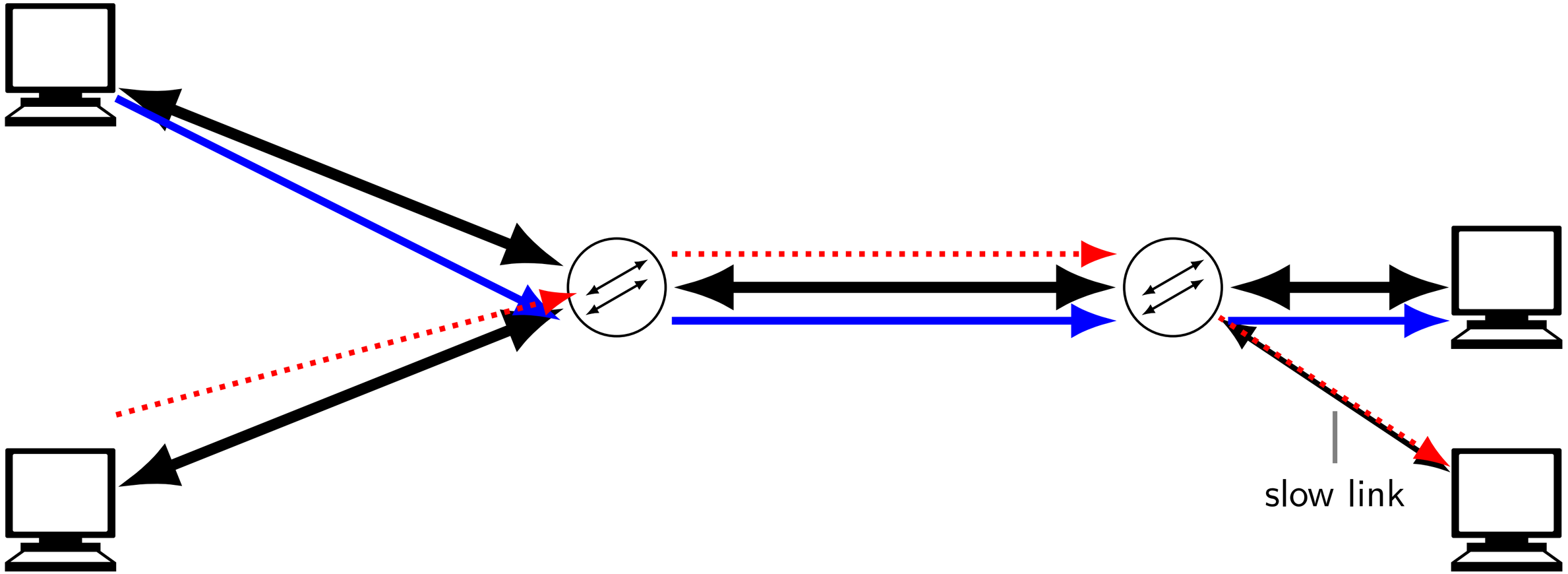
same additive increase factor (for 45 degree angle)

- TCP so far: +1 segment/round-trip-time
- round trip time, segment size may vary between flows

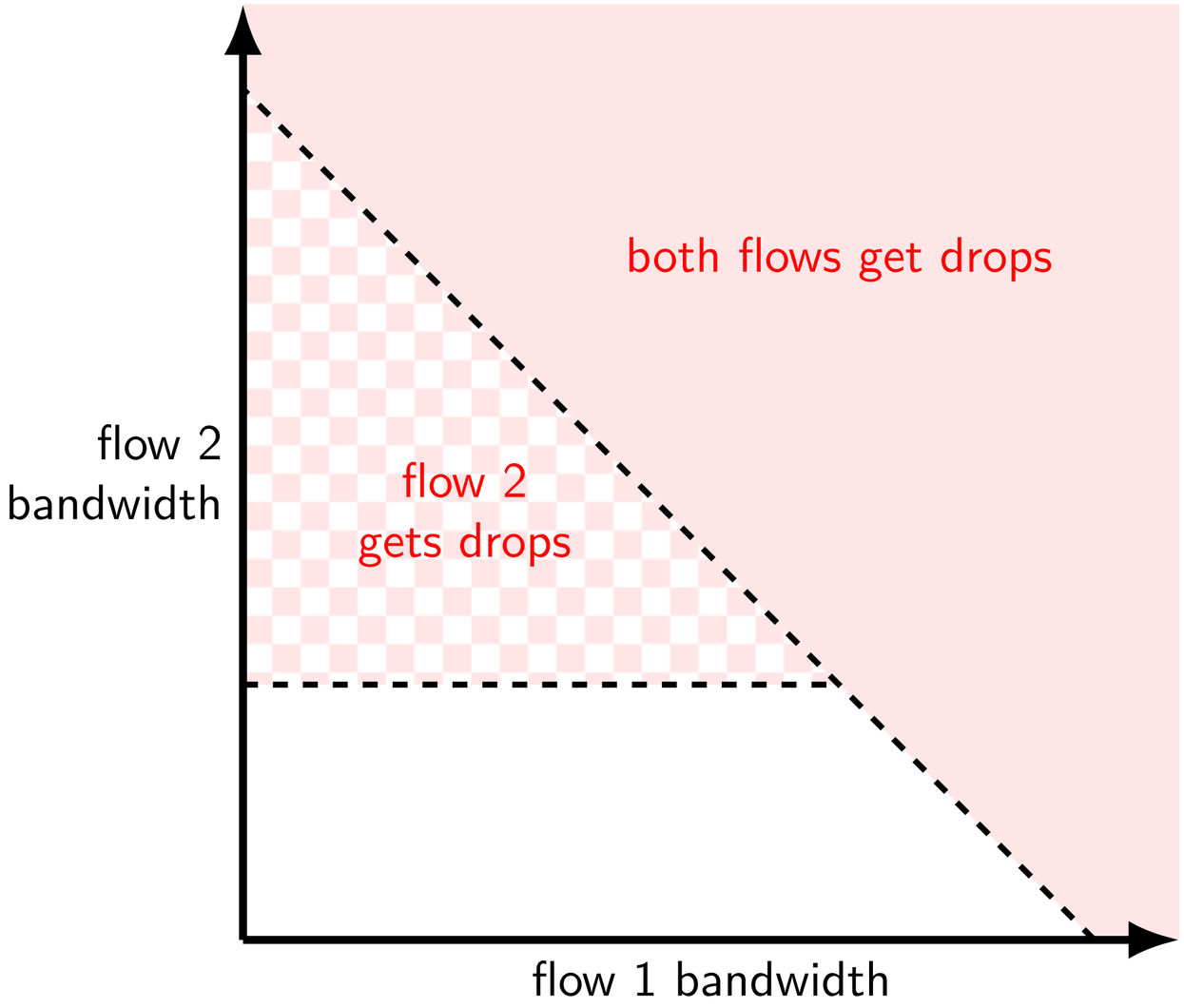
# a scenario



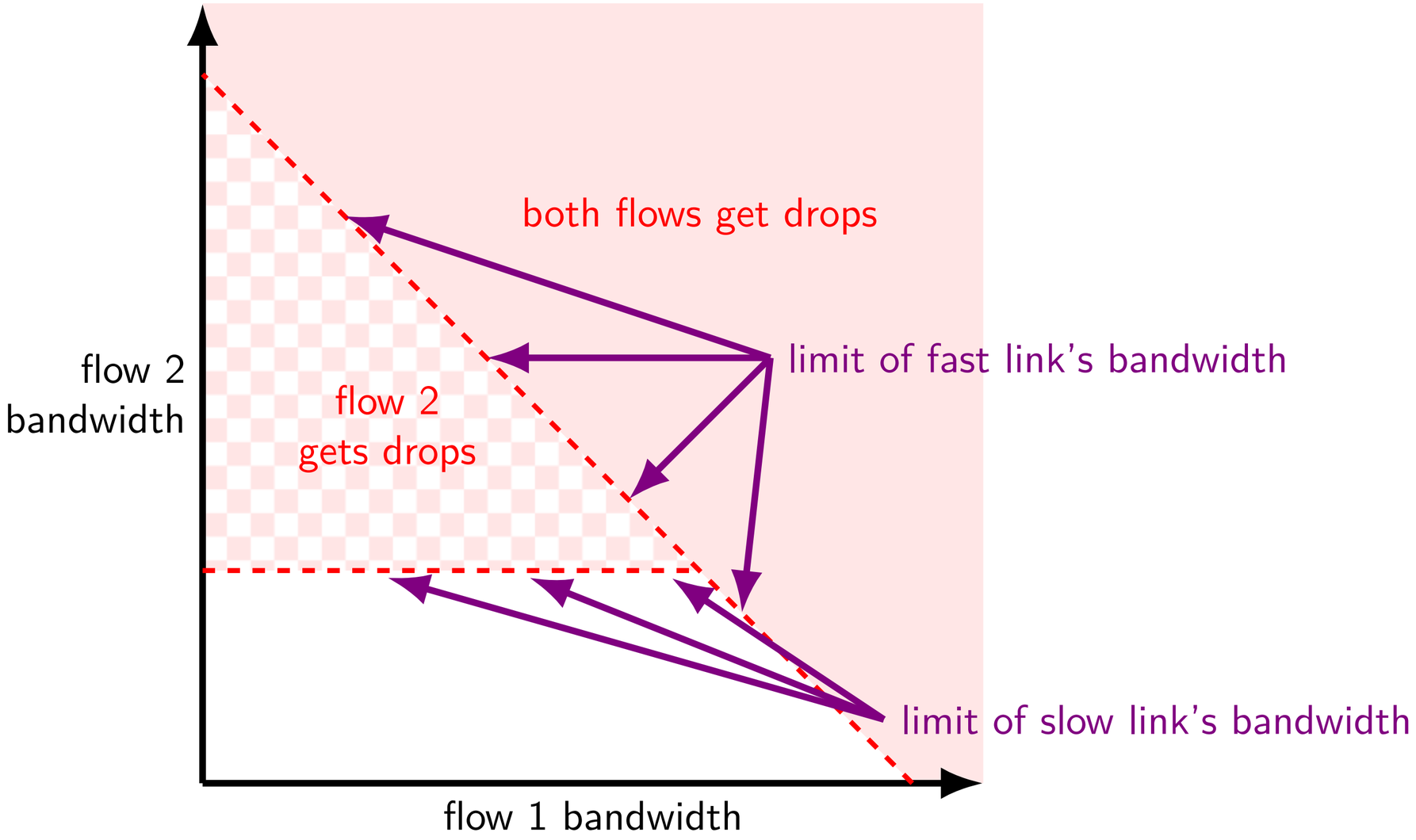
# a scenario



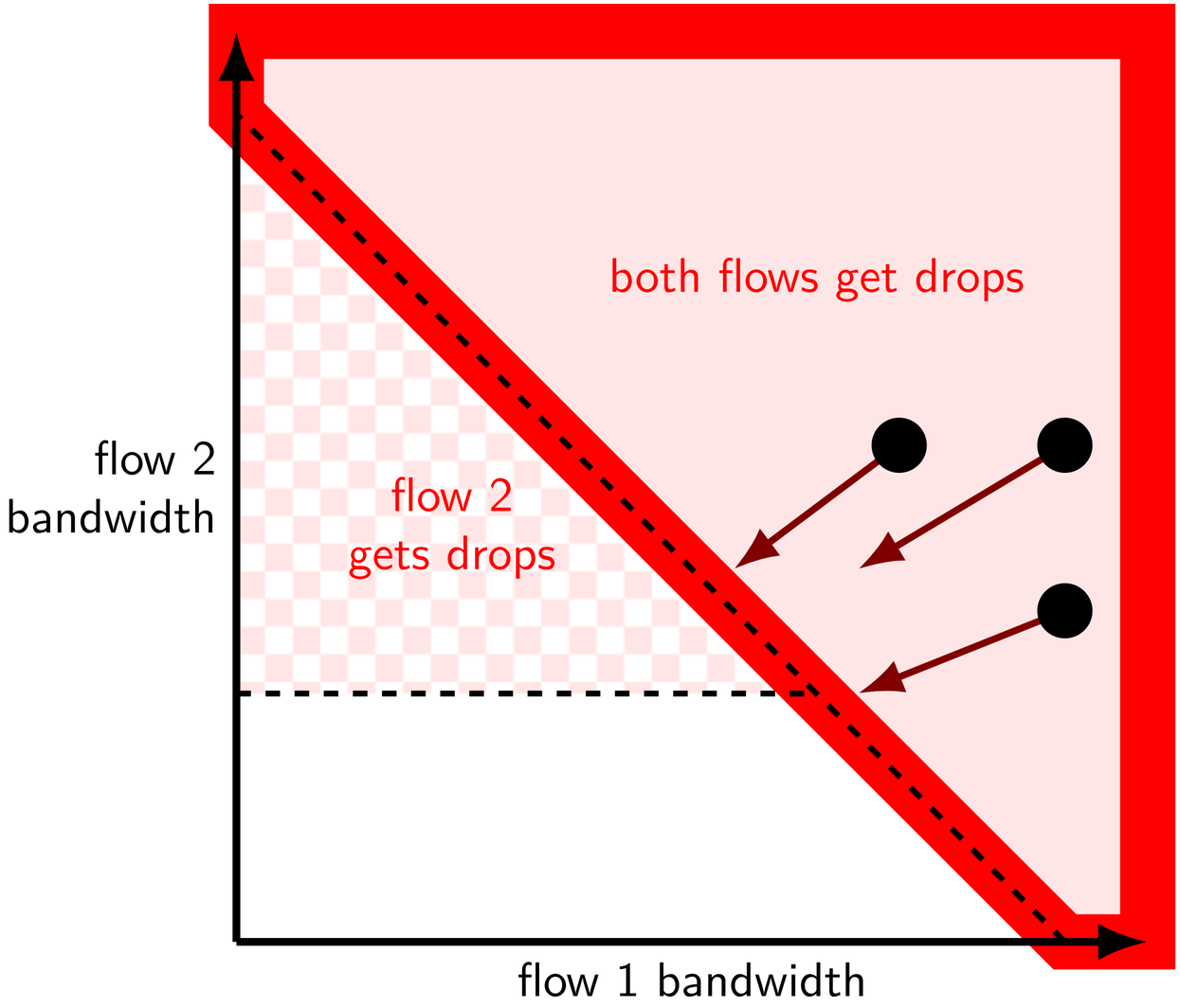
# slow link + mixed traffic



# slow link + mixed traffic

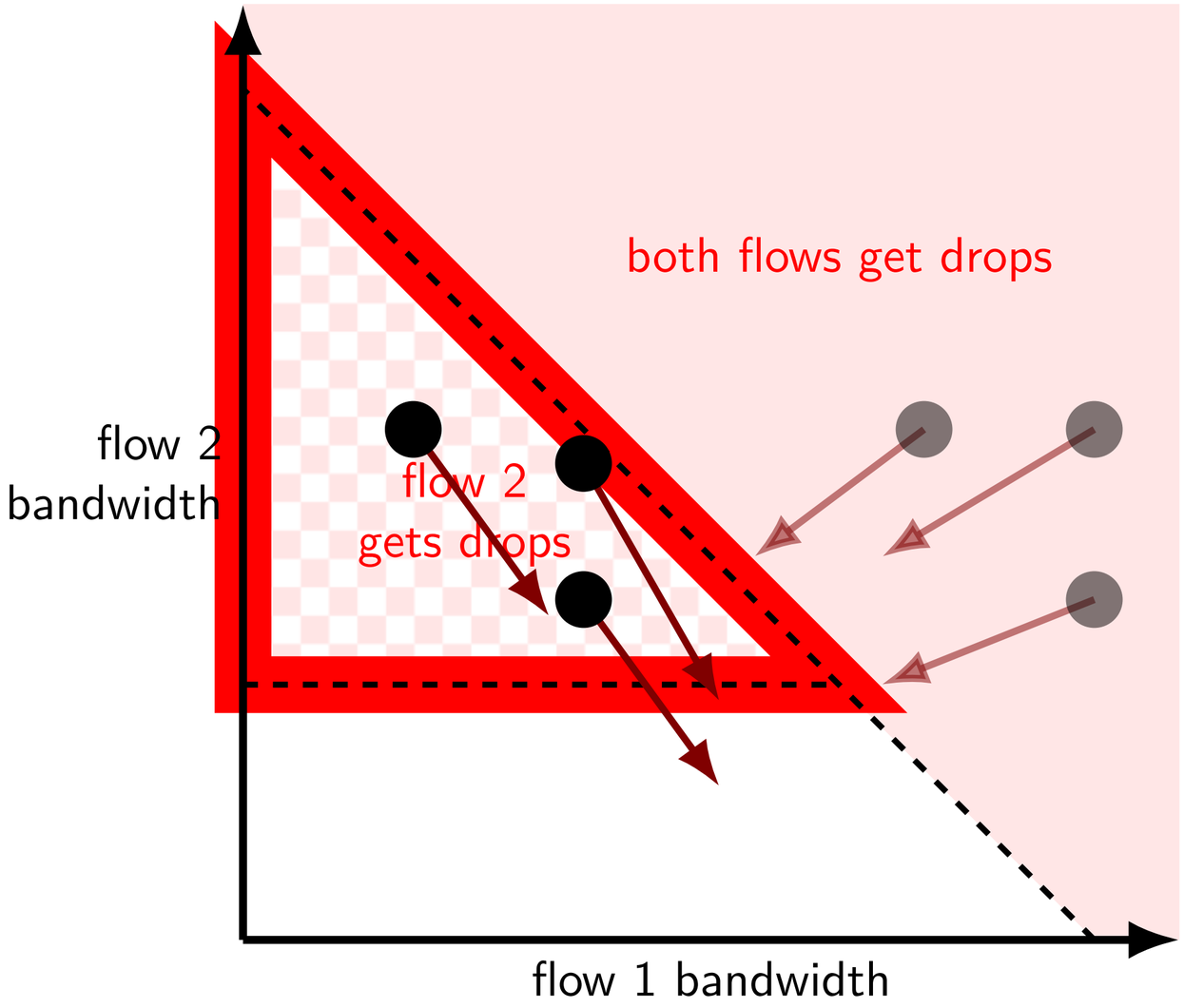


# slow link + mixed traffic



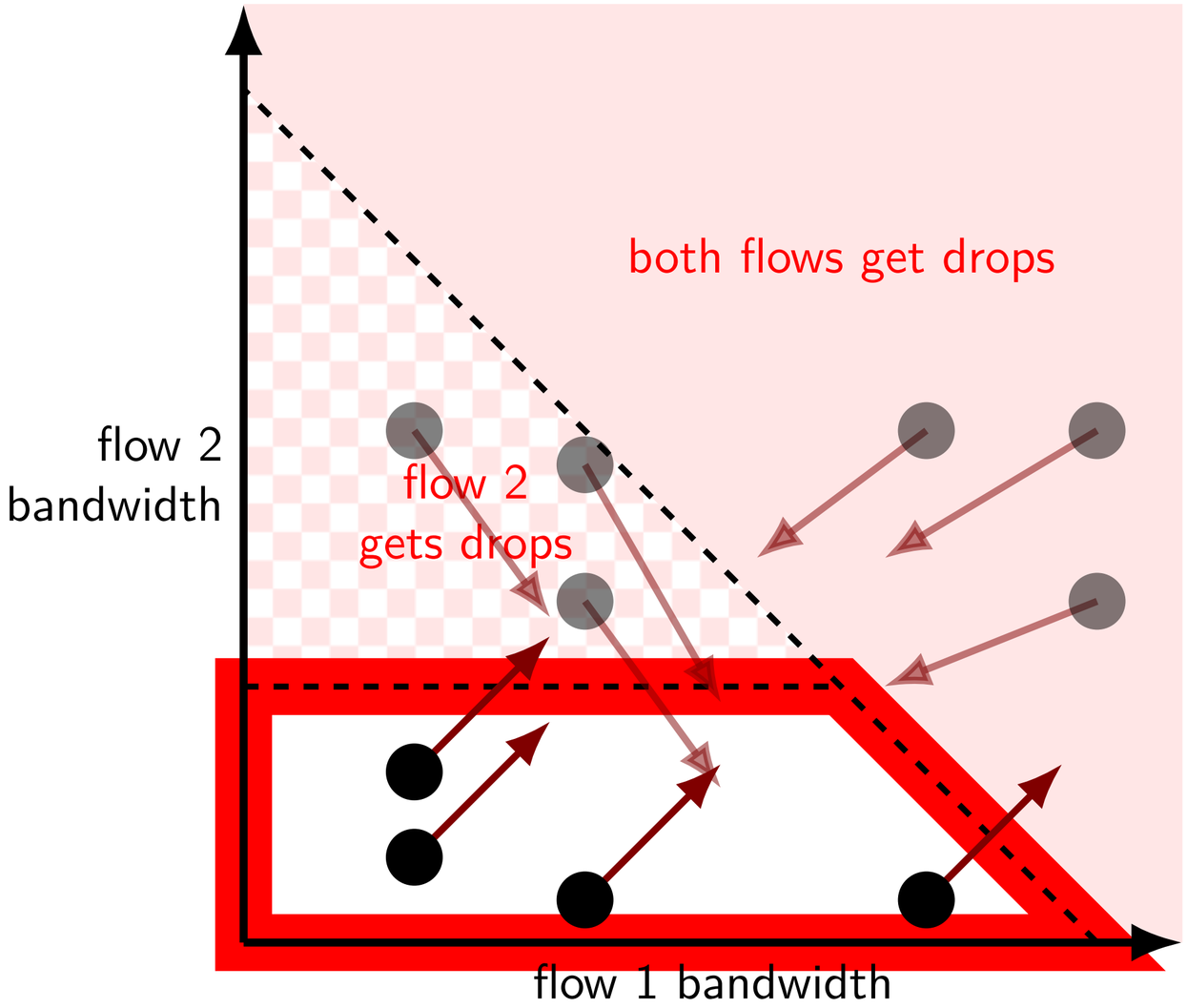

if both flows see drops,
multiplicative decrease
(toward origin)

# slow link + mixed traffic



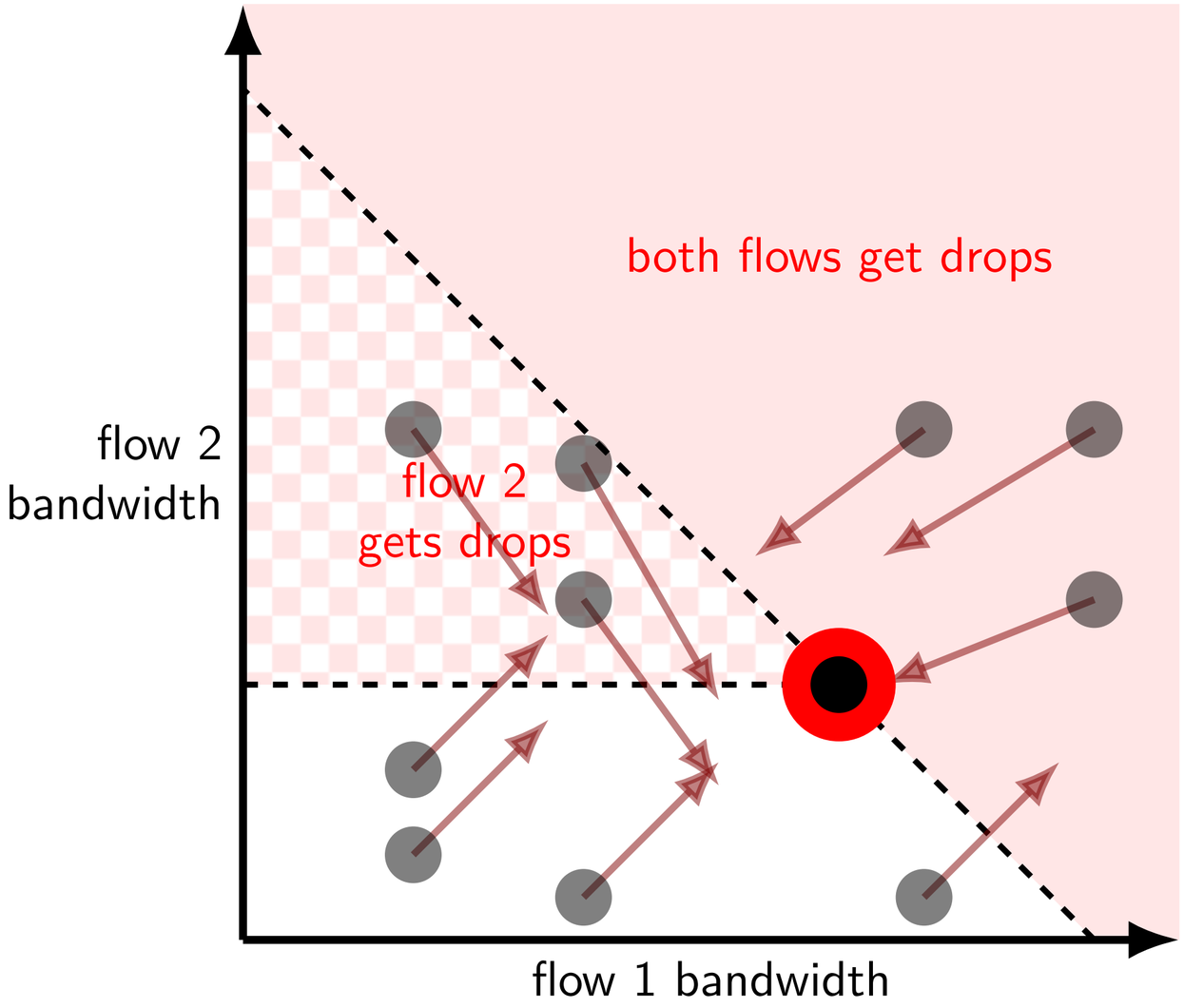

if only flow 2 see drops,
it decreases bandwidth and
flow 1 increase bandwidth

# slow link + mixed traffic




if both flows see no drops,
both increase additively
(45 degree angle)

# slow link + mixed traffic




result: flow 2 reaches
limit of slow link
flow 1 gets the rest
of the bandwidth

# fairness metrics

would like to say both allocations are ‘fair’

easy when ideal allocation is equal, but that’s not always the case

perhaps not equal, but most equal we can give on network

would like some way of formalizing this

# fairness intuition

unfair allocation = someone gets much less than others

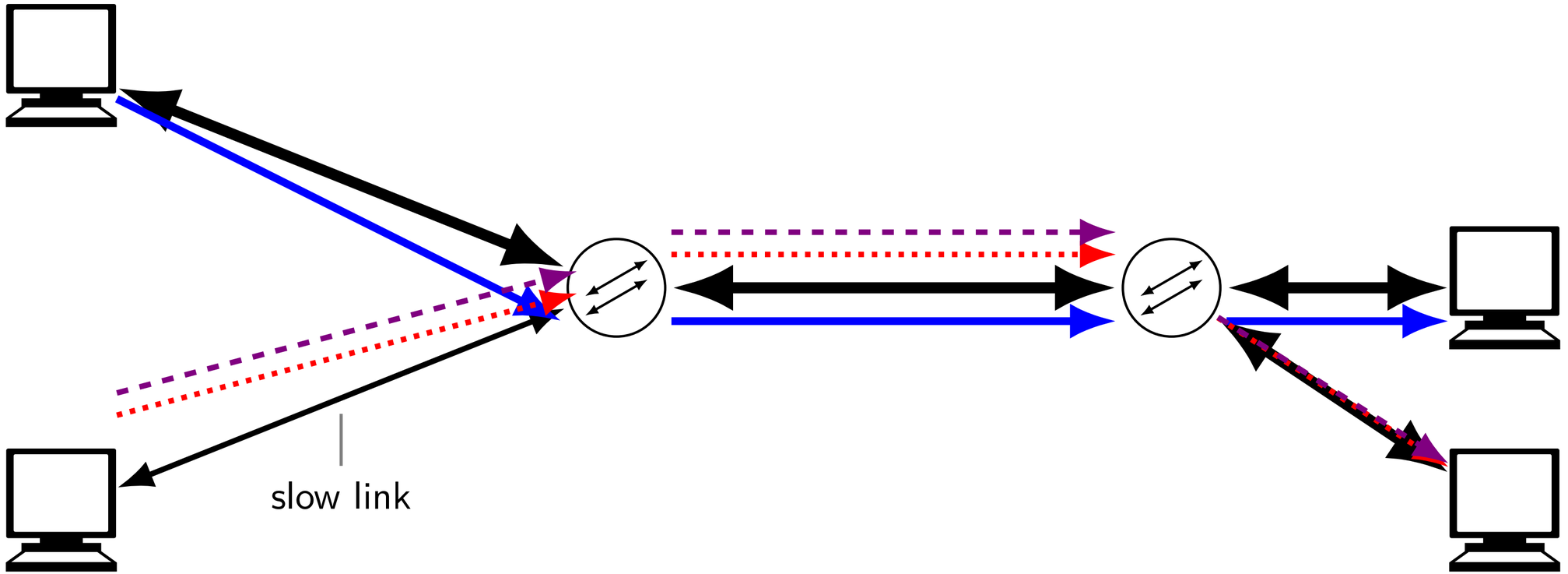
consequence: let's look for "starved" flow

if we can add to one flow…
and only hurt flows that are faster than it…
then that's "unfair"

that flow "deserves" more (possibly at expense of "richer" flows)

idea called *max-min fairness*

# exercise




let’s say slow link has 5 MByte/s capacity, other links 20MByte/s

why are these fair/unfair? (by min-max fairness)

solid = 10MByte, dotted = 2MByte/s, dashed = 3MByte/s

solid = 16MByte, dashed = 2MByte/s, dashed = 2MByte/s

# Jain's fairness index

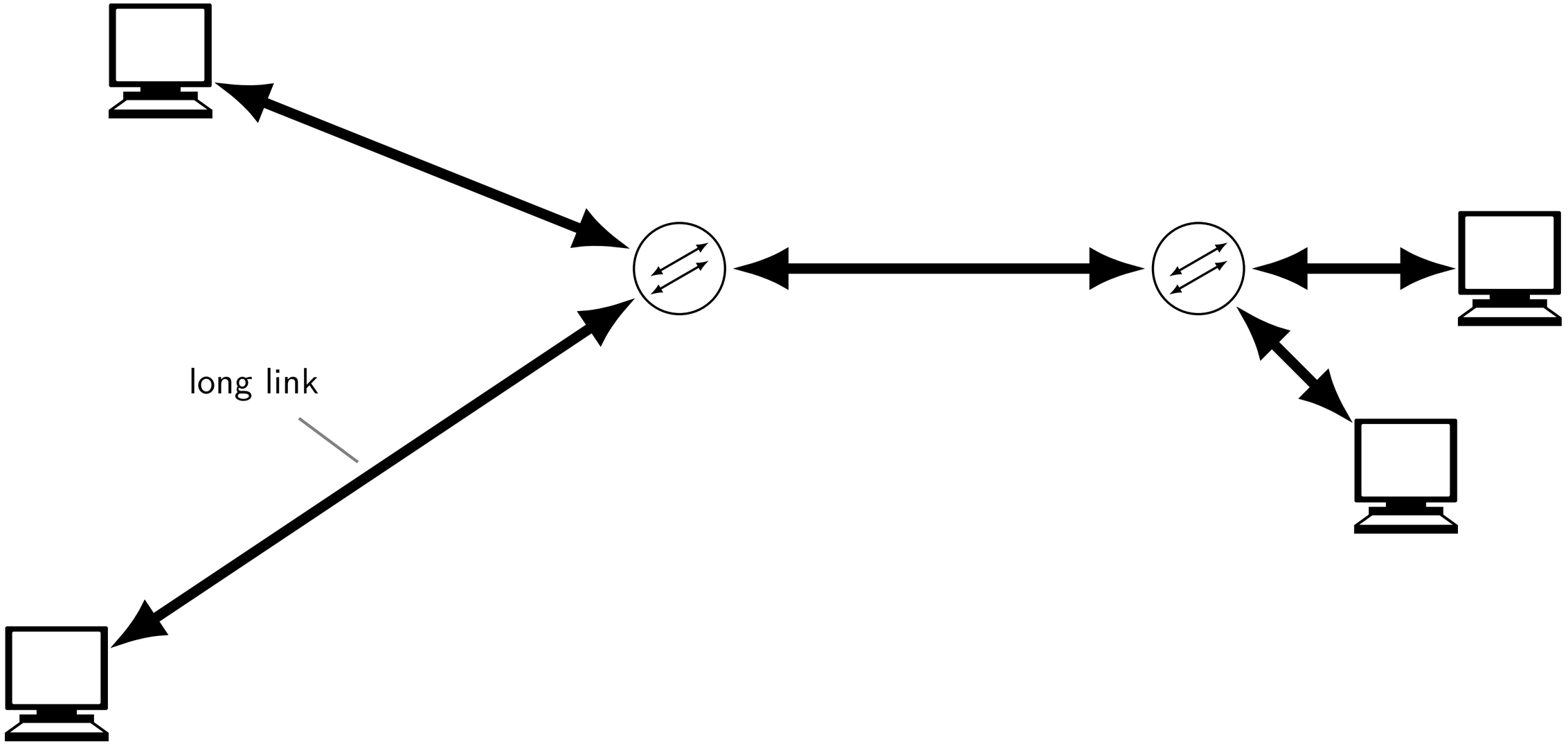
more common metric, but for scenarios where equal allocation makes sense

if $x_i$ is i'th flow's share:

$$\frac{(\sum x_i)^2}{n \cdot \sum x_i^2}$$

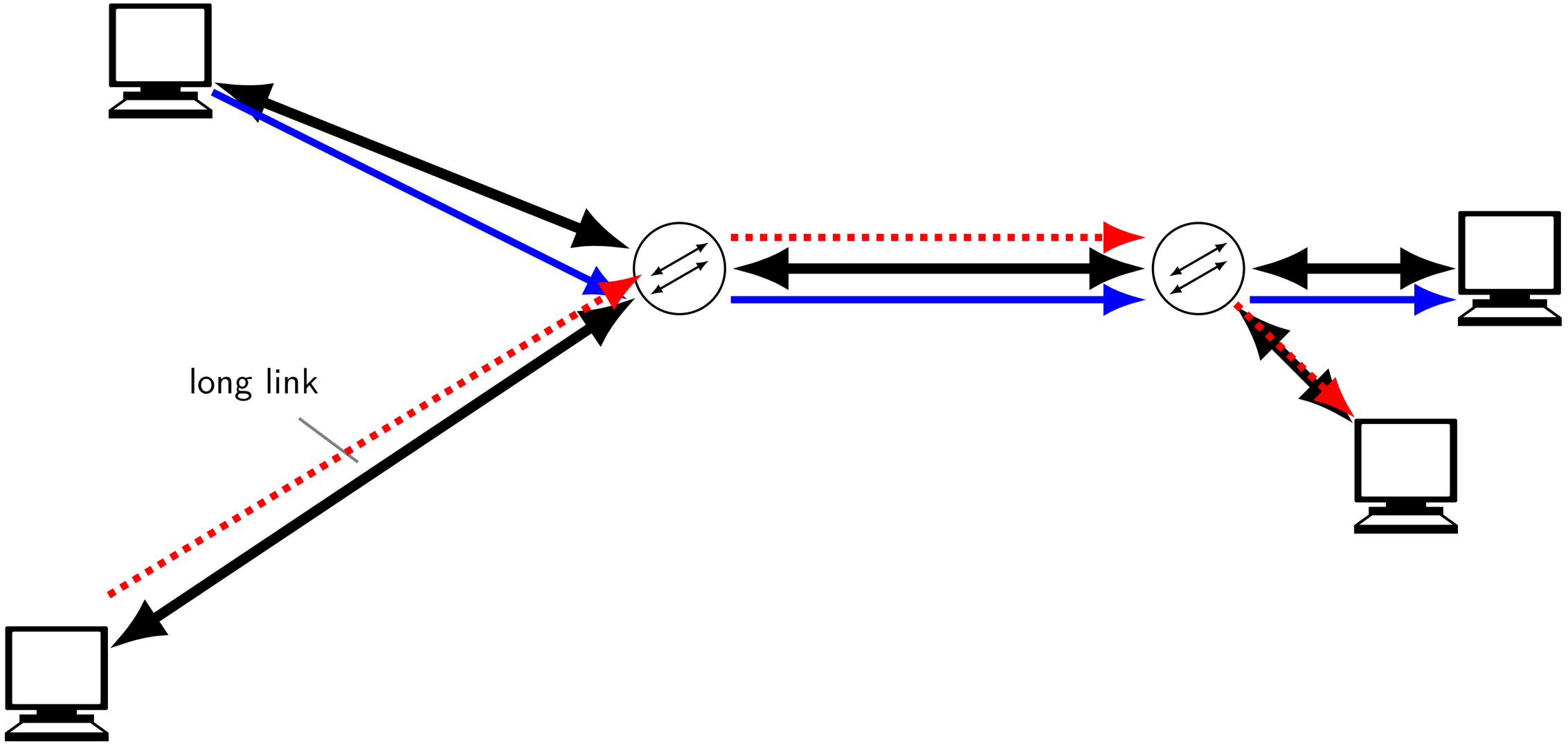
approaches 1 when allocations equal, $\frac{1}{n}$ if one flow gets everything

# long links



# long links

# long links



# exercise: window size?

flow 1: 500 packets/sec, 50 ms round trip

flow 2: 500 packets/sec, 100 ms round trip

exercise: what window size achieves this for each flow?

# exercise: window size?

flow 1: 500 packets/sec, 50 ms round trip

flow 2: 500 packets/sec, 100 ms round trip

exercise: what window size achieves this for each flow?

1: window size 25; 2: window size 50

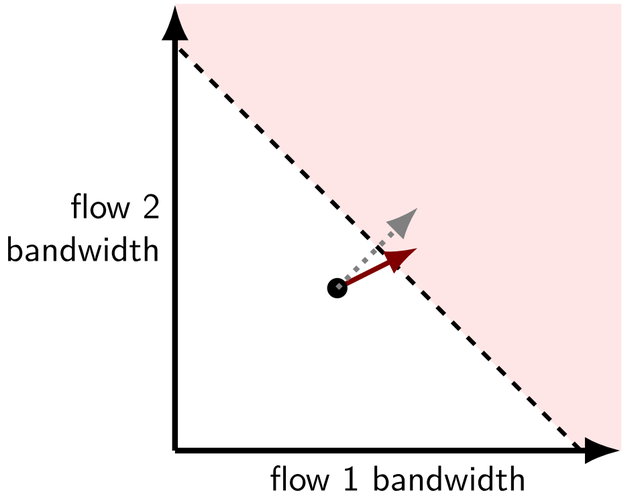
# revisiting additive increase

TCP: add +1 to window size each round trip time

flow 1: window 25+1 $\rightarrow$ 26 pkt/50 ms = 520 pkt/sec

flow 2: window 50+1 $\rightarrow$ 51 pkt/100 ms = 510 pkt/sec

using rough model from before:




flow 1 increases faster
than flow 2 increases
not 45-degree angle anymore

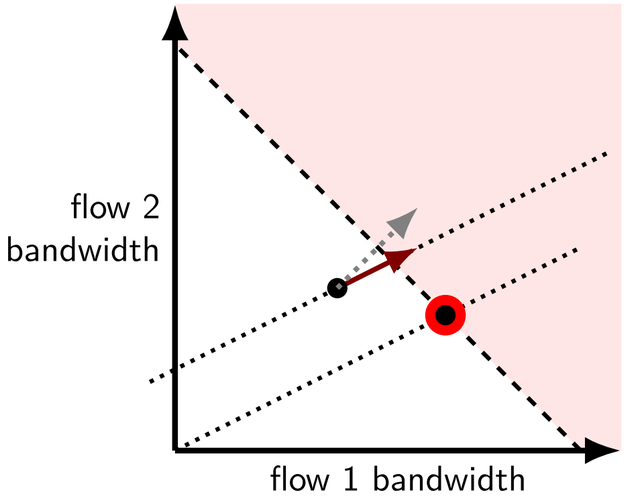
# revisiting additive increase

TCP: add +1 to window size each round trip time

flow 1: window 25+1 $\rightarrow$ 26 pkt/50 ms = 520 pkt/sec

flow 2: window 50+1 $\rightarrow$ 51 pkt/100 ms = 510 pkt/sec

using rough model from before:




in equilibrium
flow 1 gets more bandwidth

# other unfairness

lower round-trip gets more bandwidth

can also get more bandwidth by…

using more connections (‘independent’ windows)

adding more than + 1 packet to window size/RTT

# alternate congestion control

usually *more aggressive* than "normal" TCP

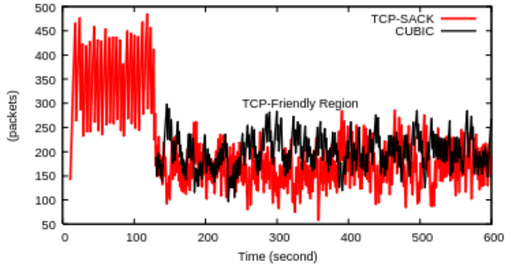
- examples we'll talk about later: CUBIC, Compound TCP
- (common defaults on recentish OSes)
- needed to fully utilize bandwidth on fast network

concern: are these "stealing" all the bandwidth from normal TCP?

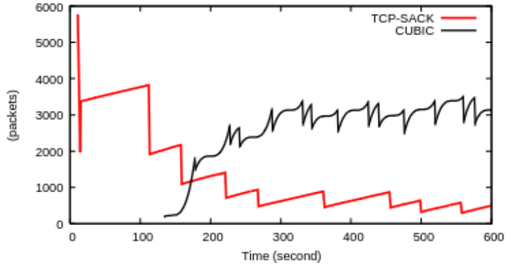
"TCP-friendly"

- should not make TCP used alongside them behave poorly

# examples: TCP-unfriendliness




(a) RTT 8ms.




(b) RTT 82ms.

**Figure 5: One CUBIC flow and one TCP-SACK flow. Bandwidth is set to 400Mbps.**

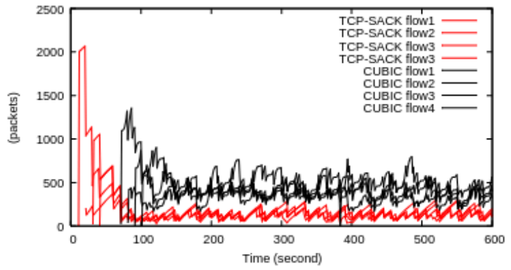



**Figure 6: Four TCP-SACK flows and four CUBIC flows over 40ms RTT**

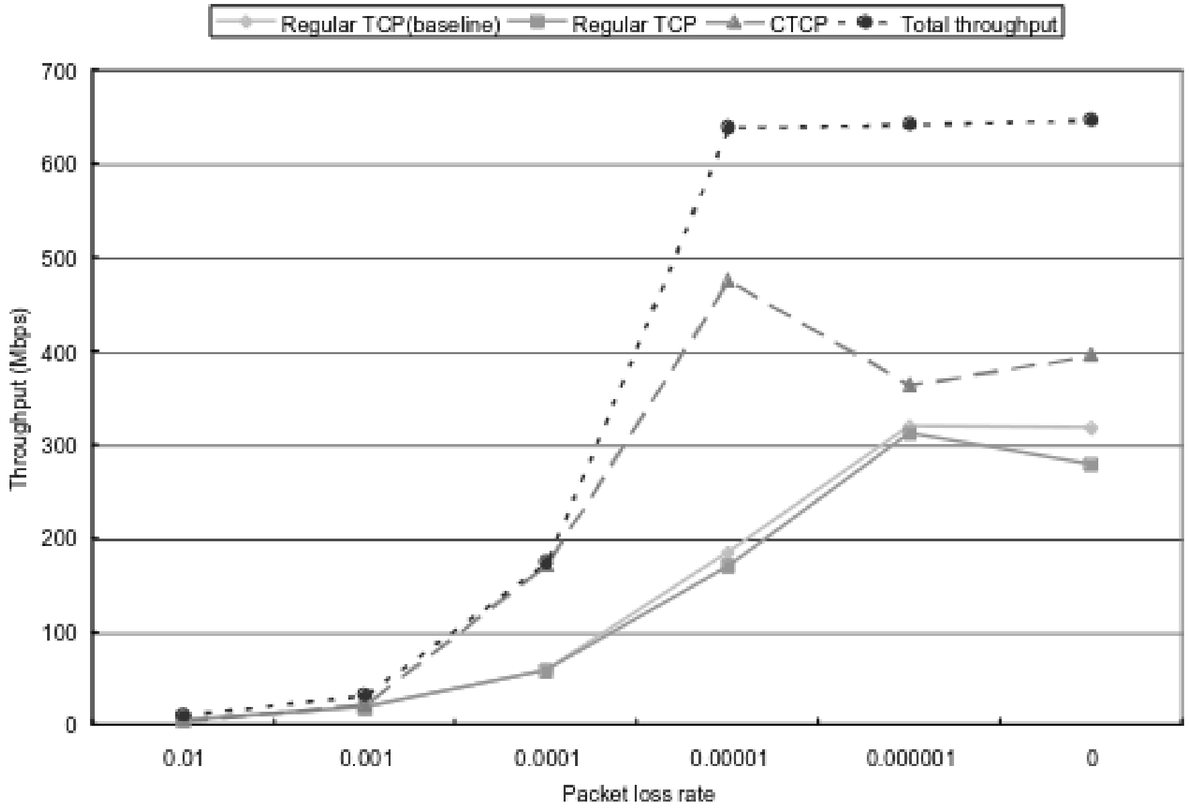



Figure 11. Throughput of CTCP and Regular TCP flows when competing for same bottleneck.

from Ha et al, “CUBIC: A New TCP-Friendly High-Speed TCP Variant”

from Tan et al, “A Compound TCP Approach for High-speed and Long Distance Networks”

# a theoretical result

for RTT-unfairness, standard TCP with selective acknowledgments:

(from Mathis et al, “The Macroscopeic Behavior of the TCP Congestion Avoidance Algorithm” (1997))

$$\text{throughput} \approx \text{constant} \times \frac{\text{packet size}}{\text{RTT}\sqrt{\text{loss rate}}}$$

# empirical results

some results from Philip (IMC'21)*

with same congestion control algorithm + RTT, Jain's fairness index > 0.99

CUBIC takes 70-80% of throughput when competing with equal number of traditional TCP flows

- cubic notably uses non-additive increase

*Philip, Ware, Athapathu, Sherry, Sekar, "Revisting TCP Congestion Control Throughput Models & Fairness Properties At Scale" (IMC'21)

# backup slides

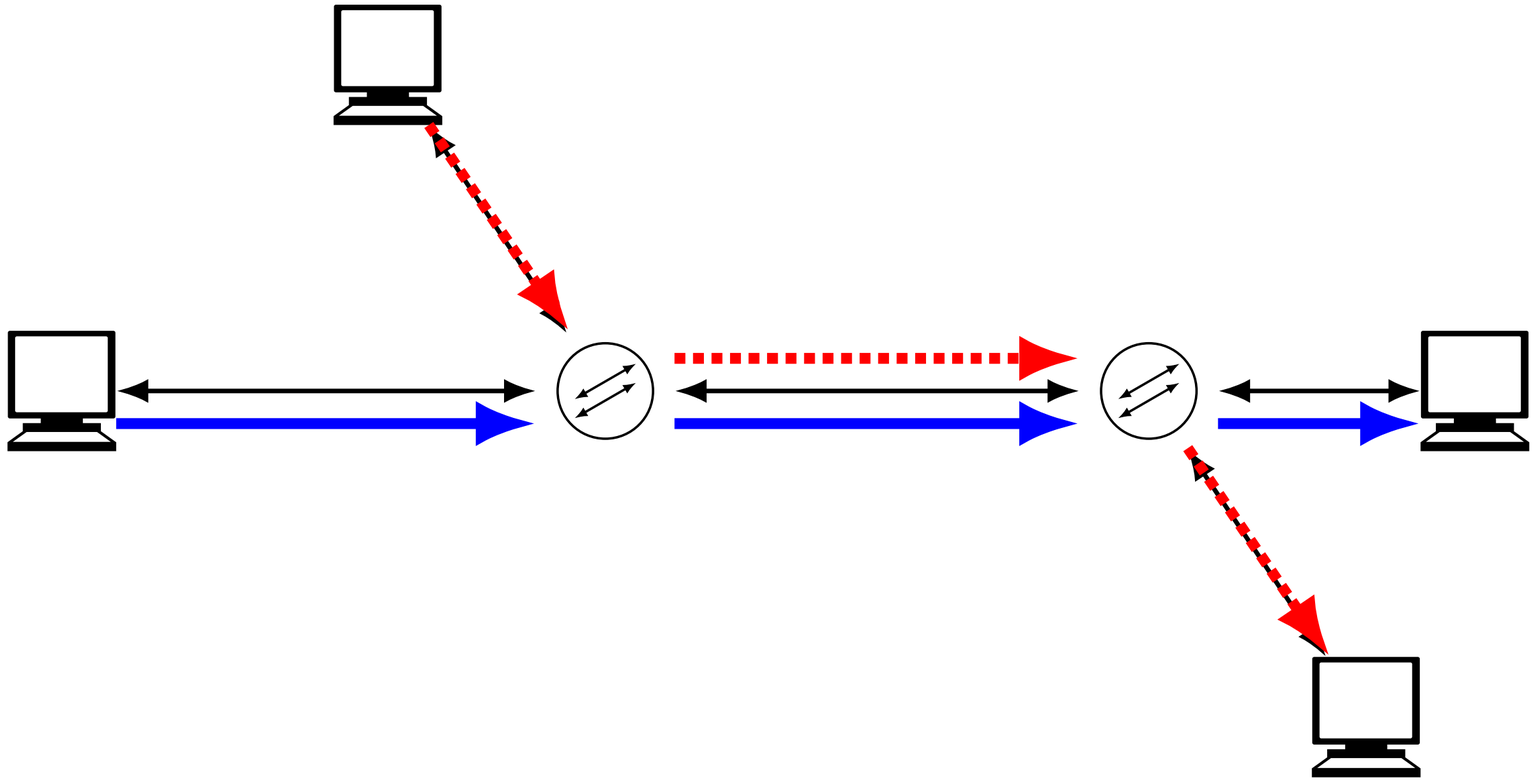
# congestion: sharing

want to consider multiple flows

key questions:

- is it stable if both flows changing window sizes?
- is there one winner/loser?
- is the winner/loser who we want it to be?

# exericse: what should happen?



two connections on shared link

intuition: each should get half of link bandwidth

question: will this happen if they don’t start equal?

# showing AIMD

slides based on Chiu and Jain, “Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks”’

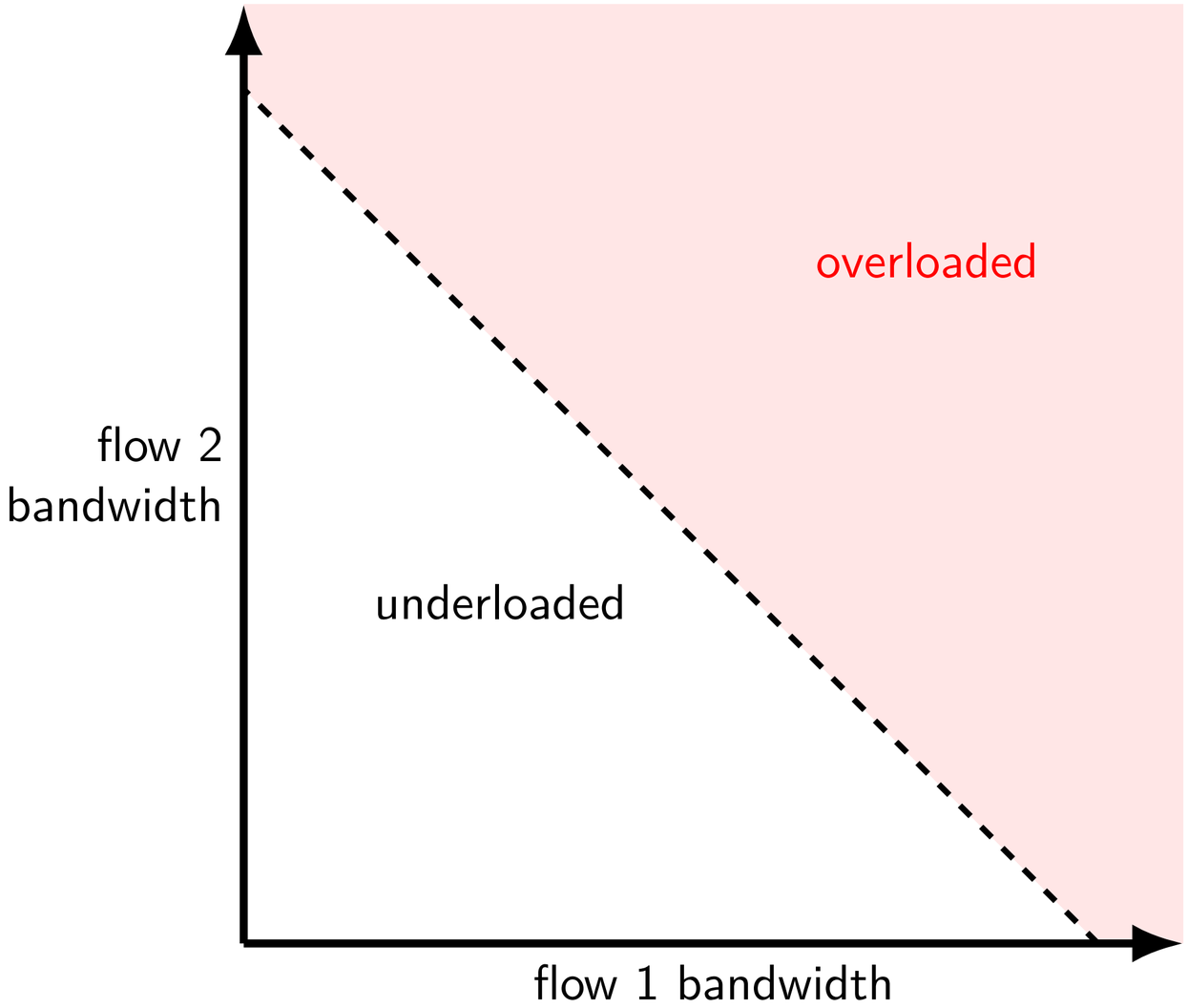
1989 paper

you might notice 1989 is well after TCP was in use

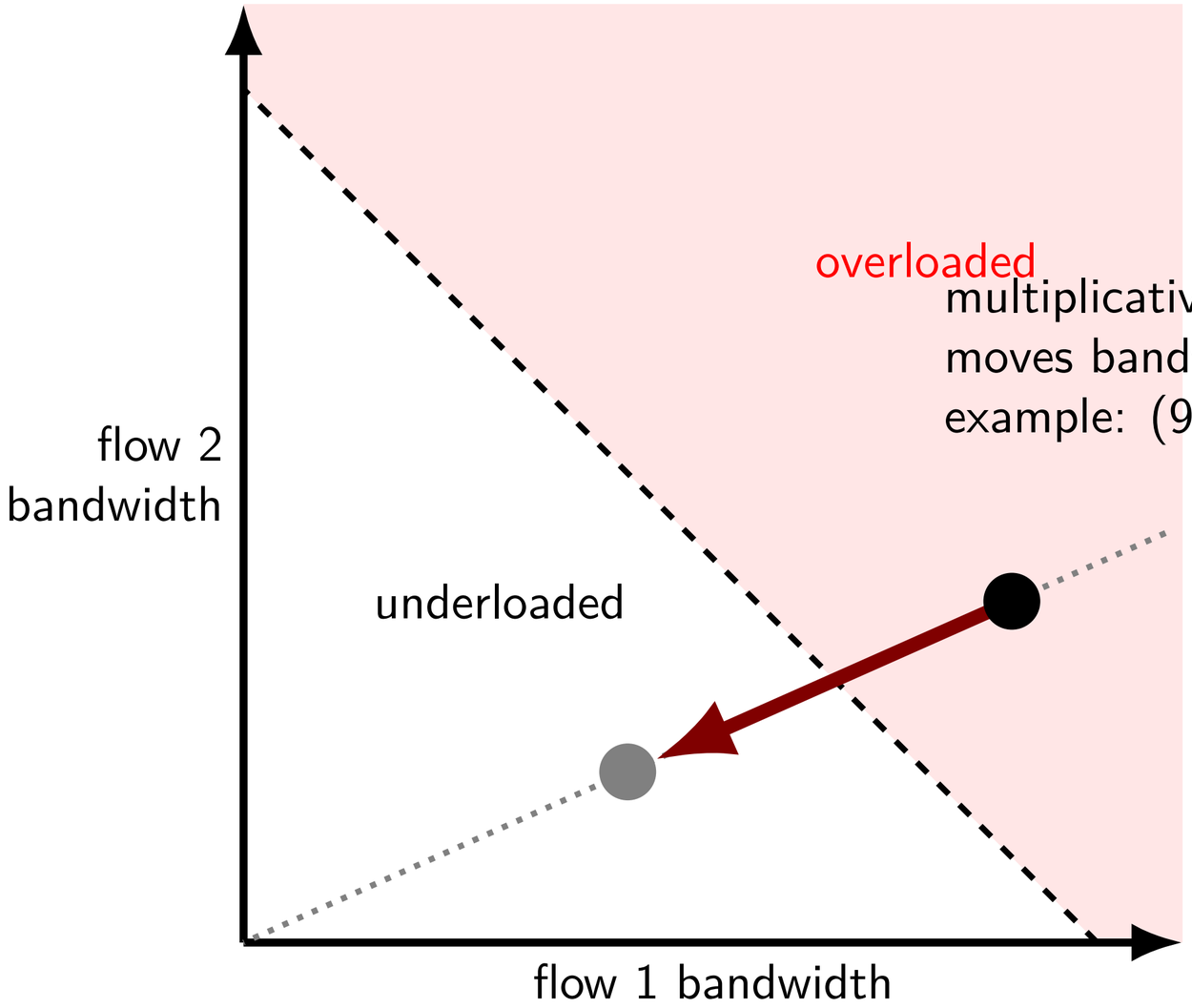
(kinda deployed without all the theory being developed…

… and it’s still not really a solved problem)

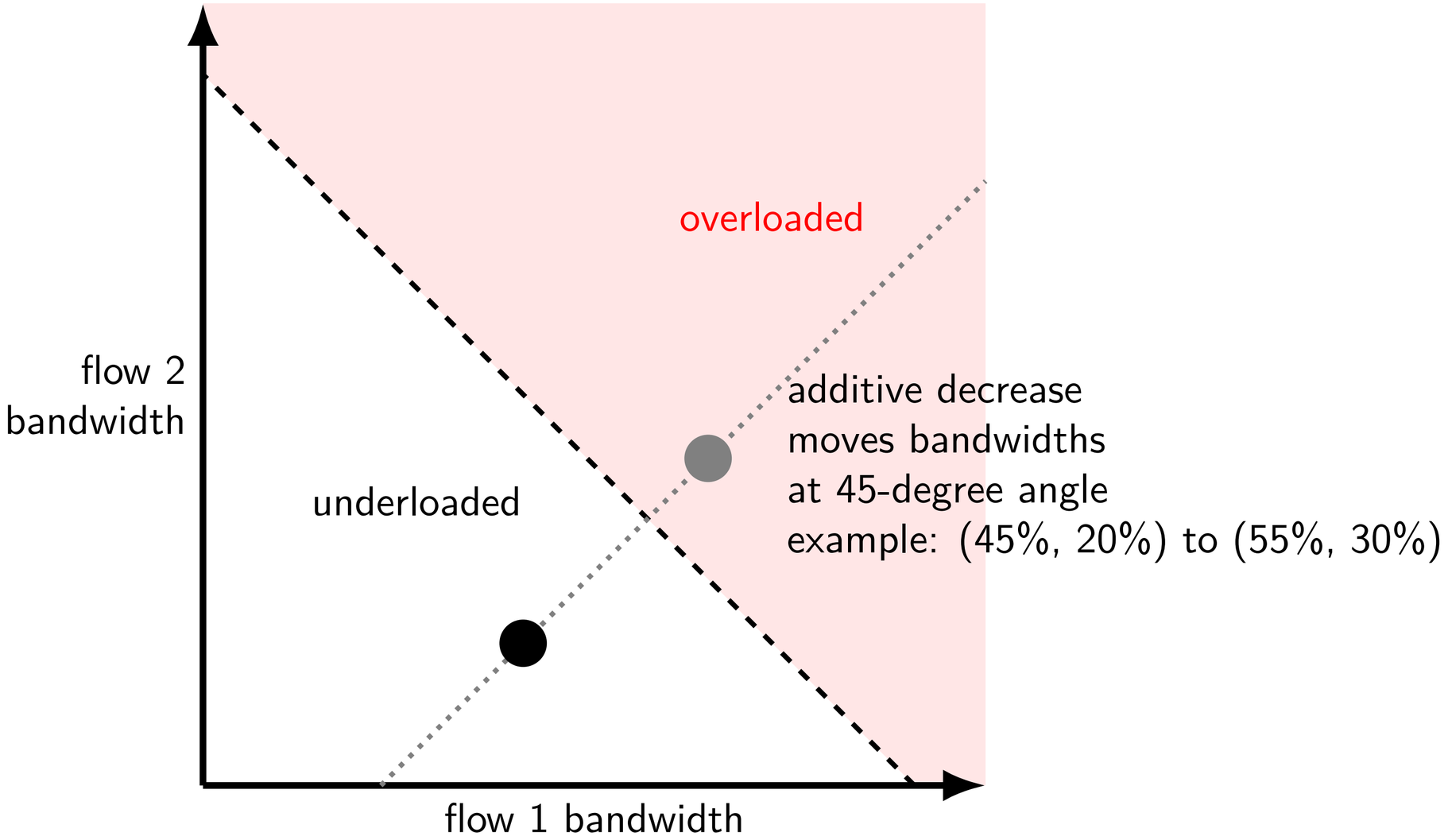
# picturing sharing



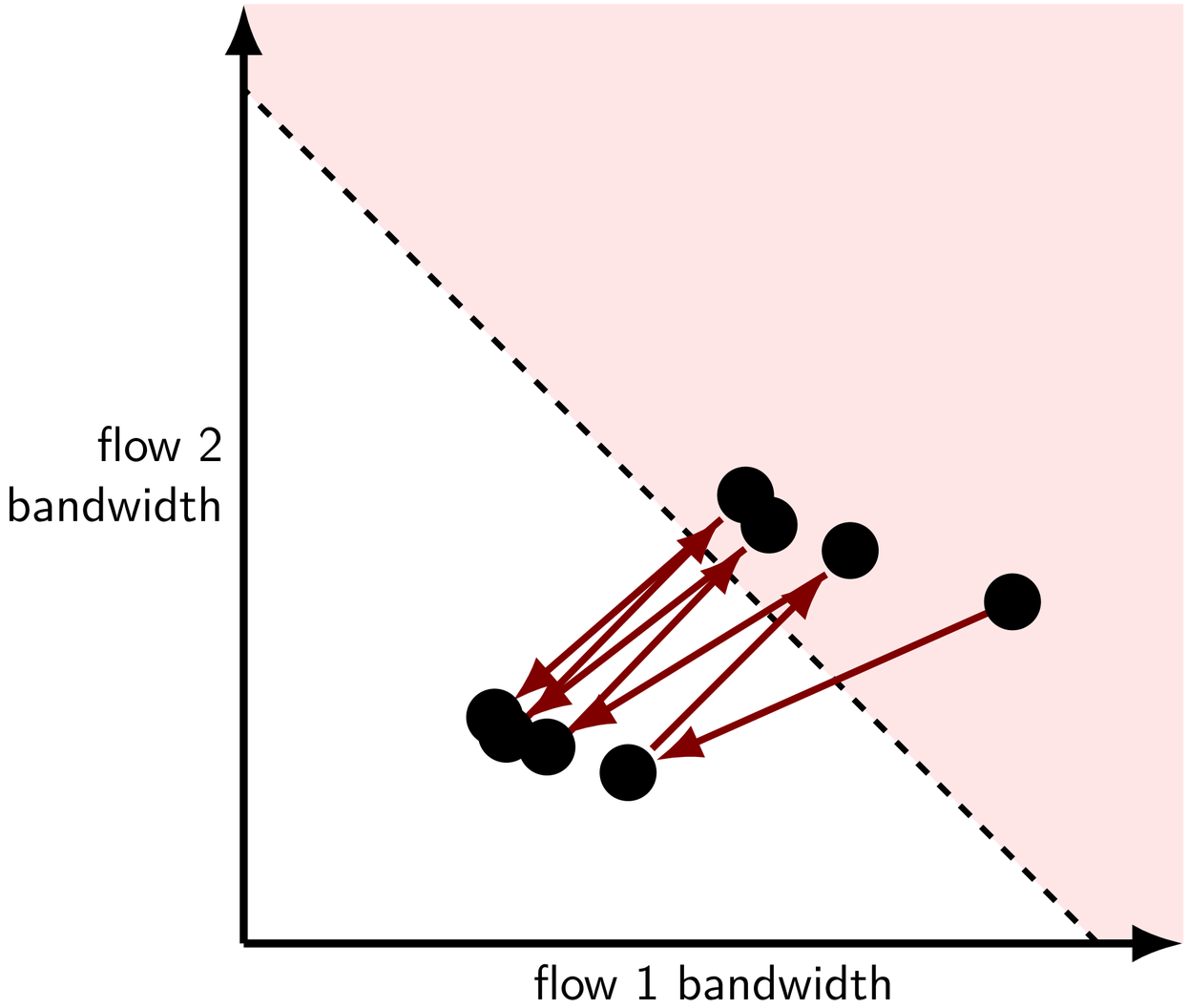
# picturing sharing



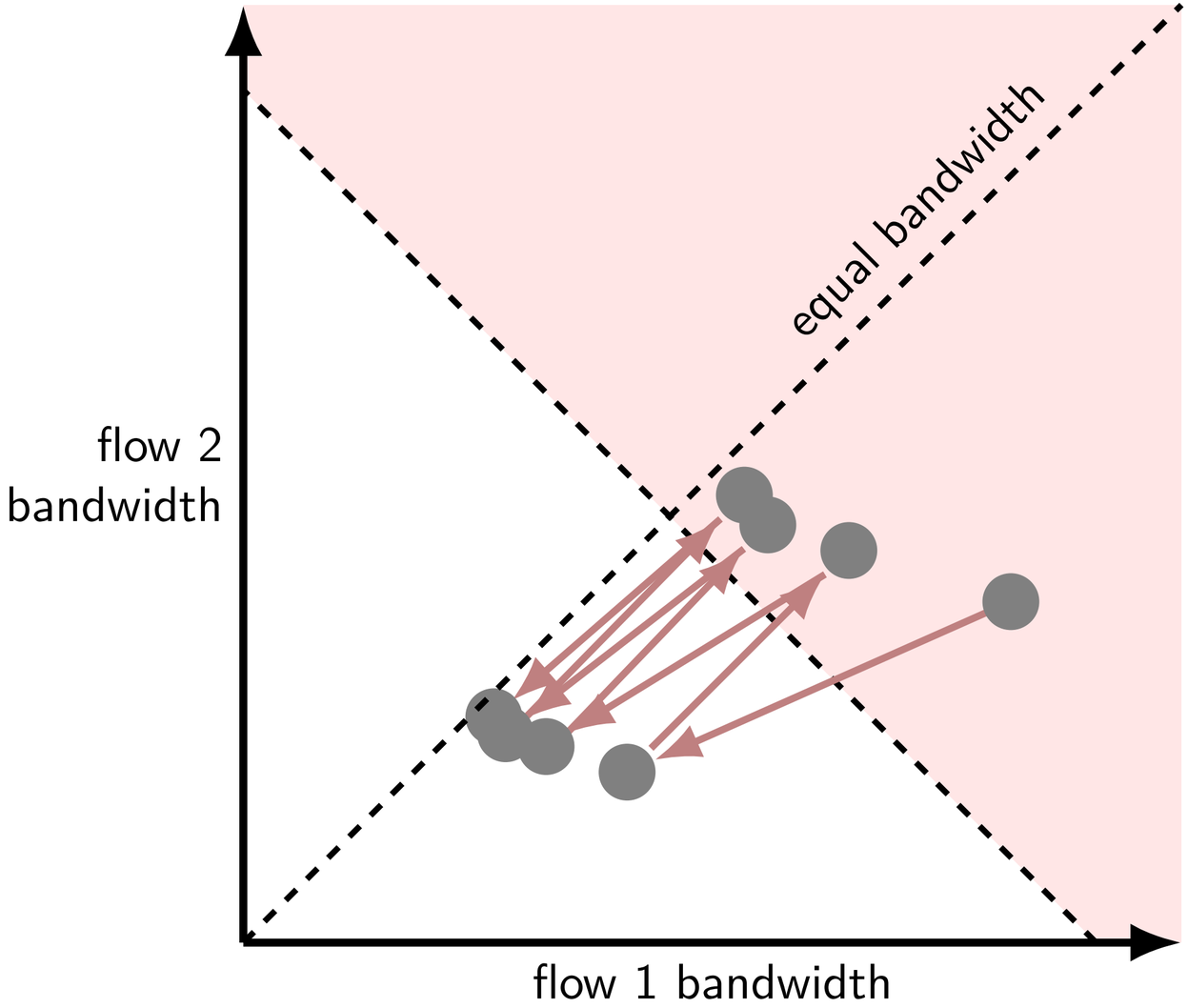
# picturing sharing



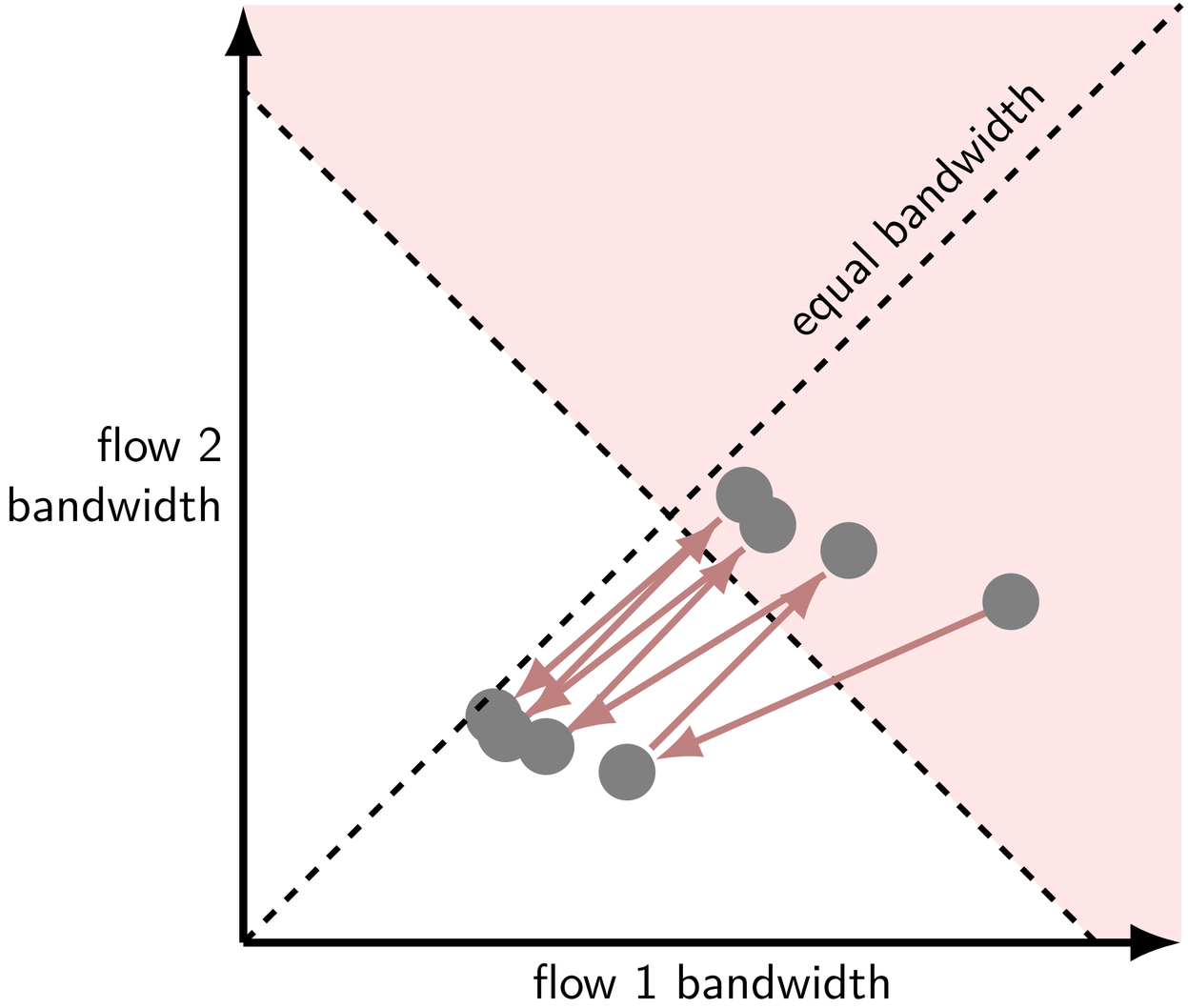
# picturing sharing



# picturing sharing



# picturing sharing




multiplicative decrease brings closer to equal bandwidth line

additive increase keeps same distance from line

# some assumptions we made

… that are probably not true

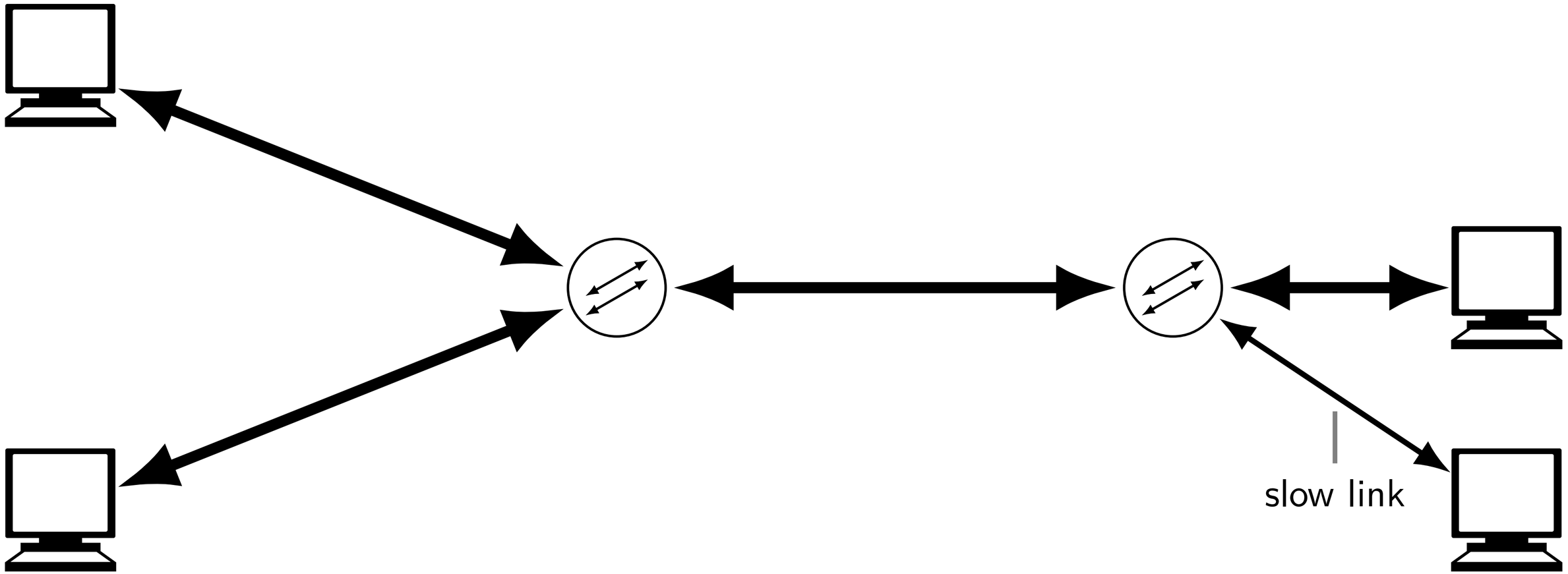
both flows experience drops equally when network overloaded

- higher-bandwidth flow more likely to see drops?
- depends on how switches manage queues?
- depends on how ‘bursty’ hosts are?

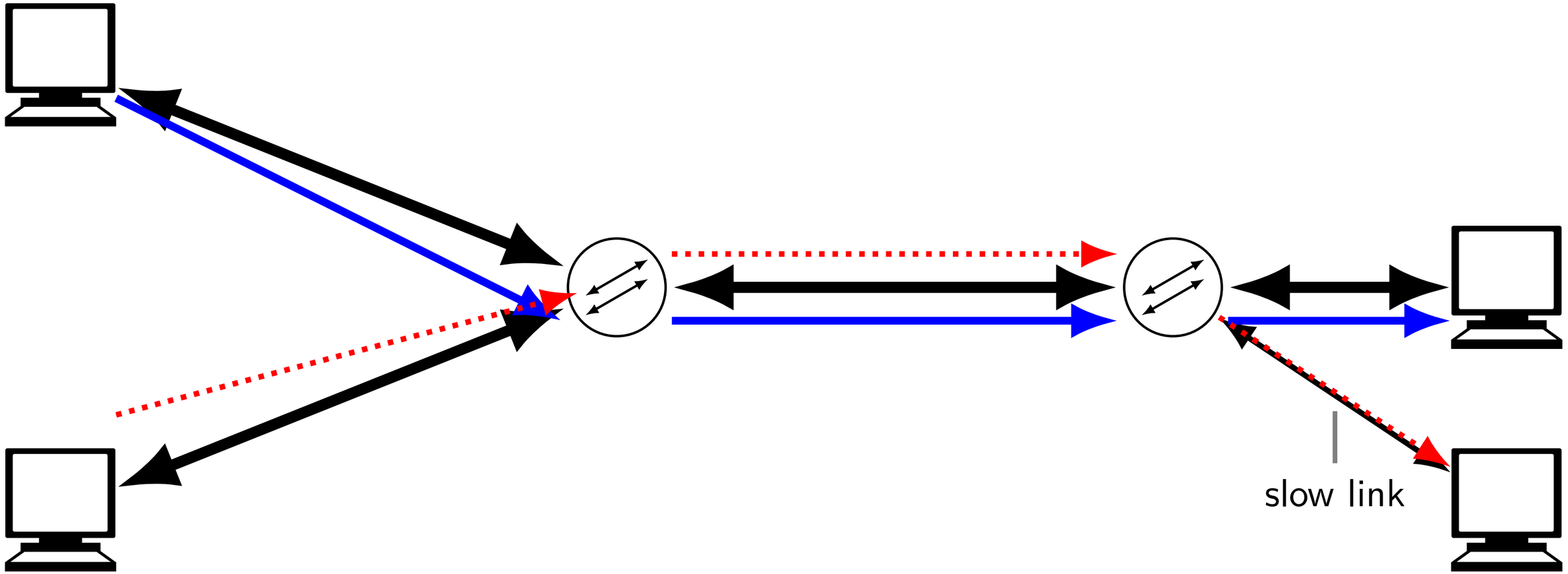
same additive increase factor (for 45 degree angle)

- TCP so far: +1 segment/round-trip-time
- round trip time, segment size may vary between flows

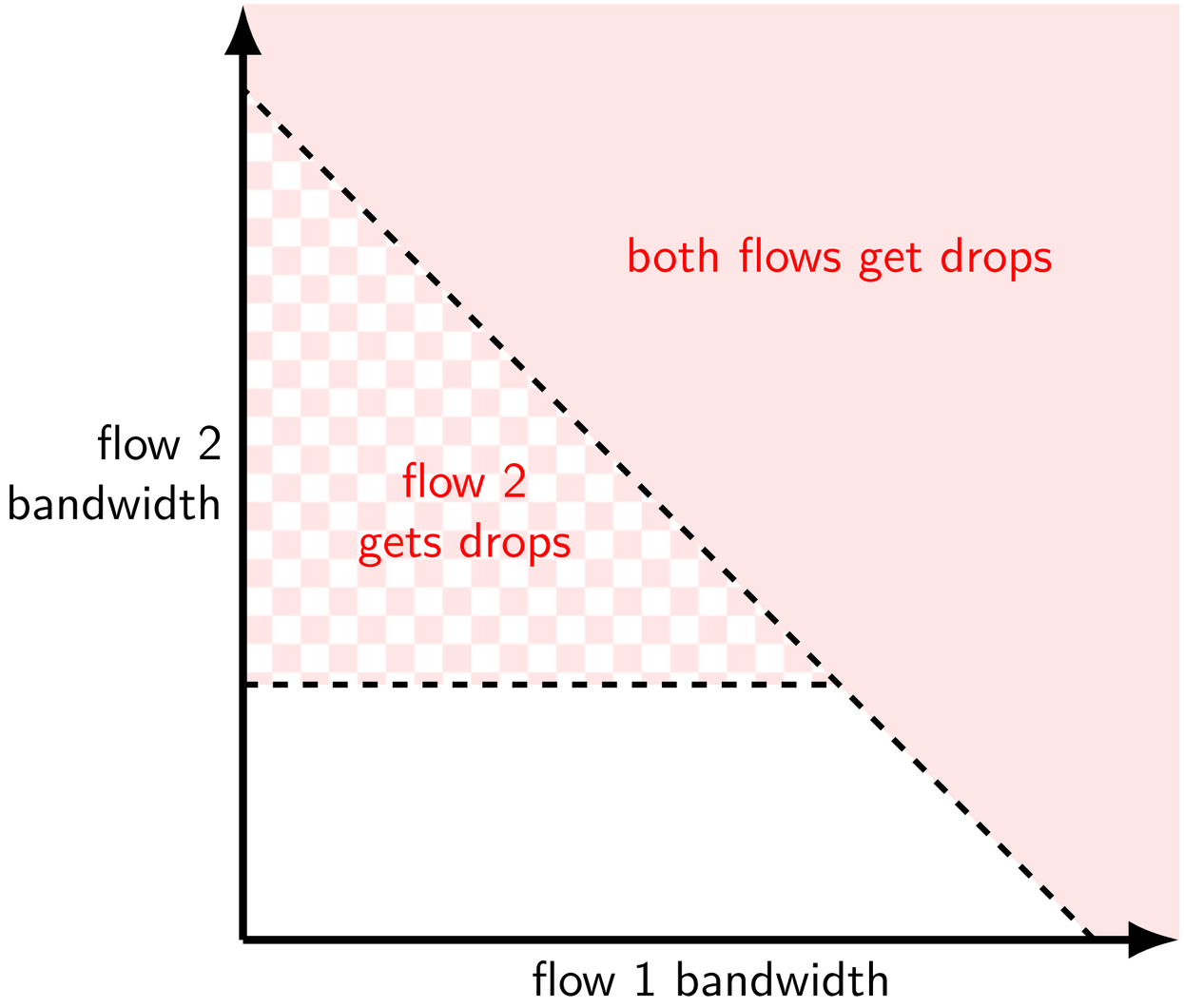
# a scenario



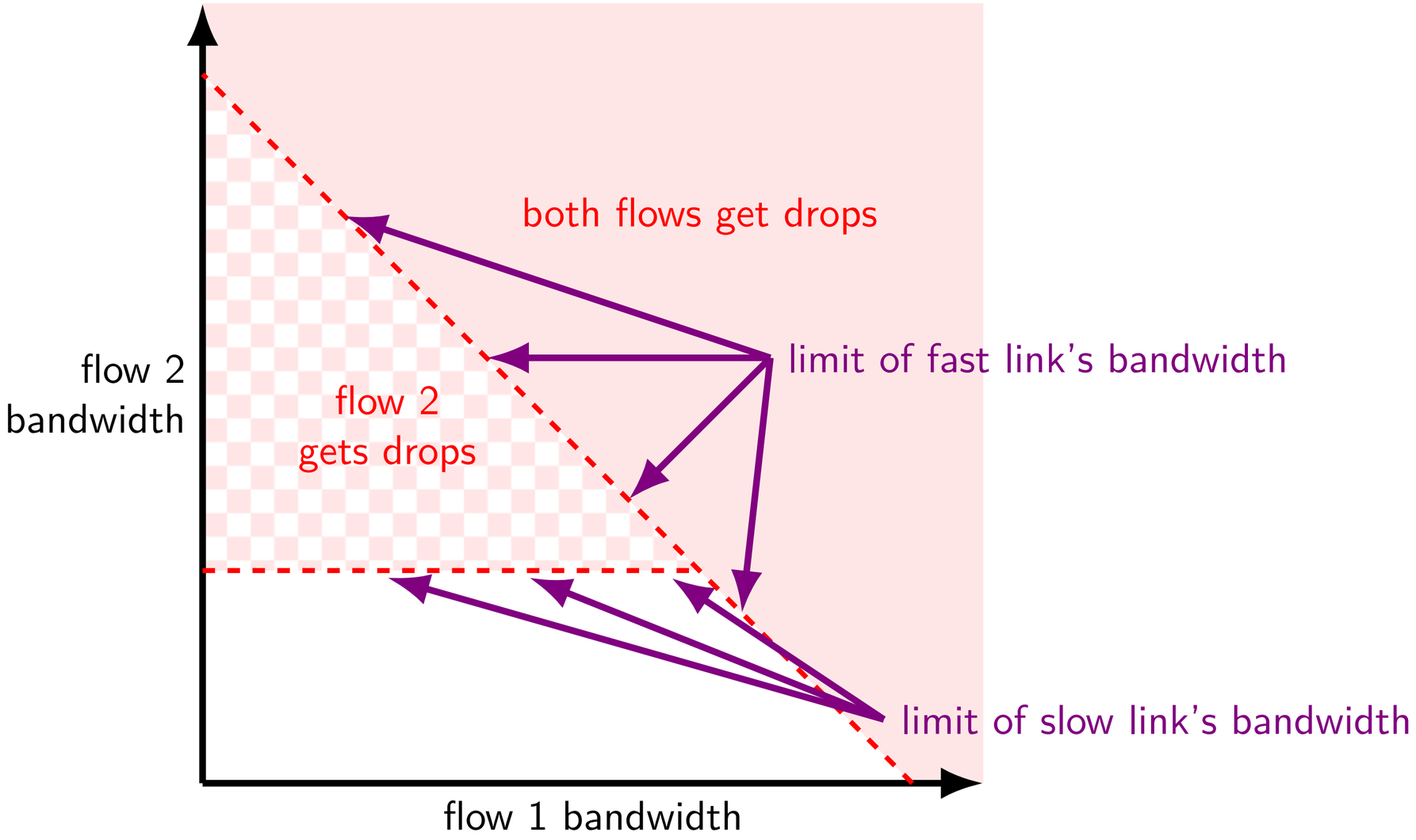
# a scenario



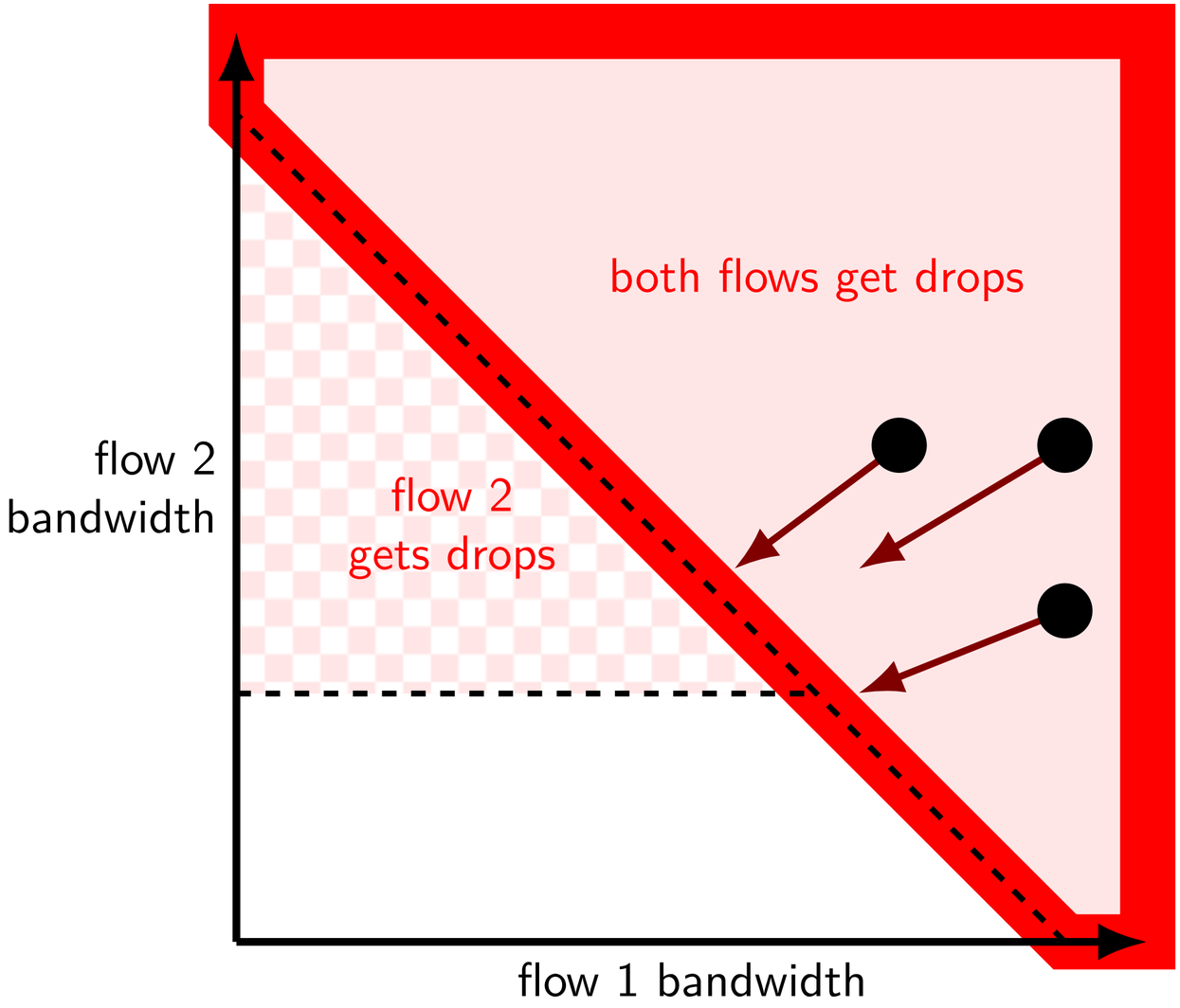
# slow link + mixed traffic



# slow link + mixed traffic

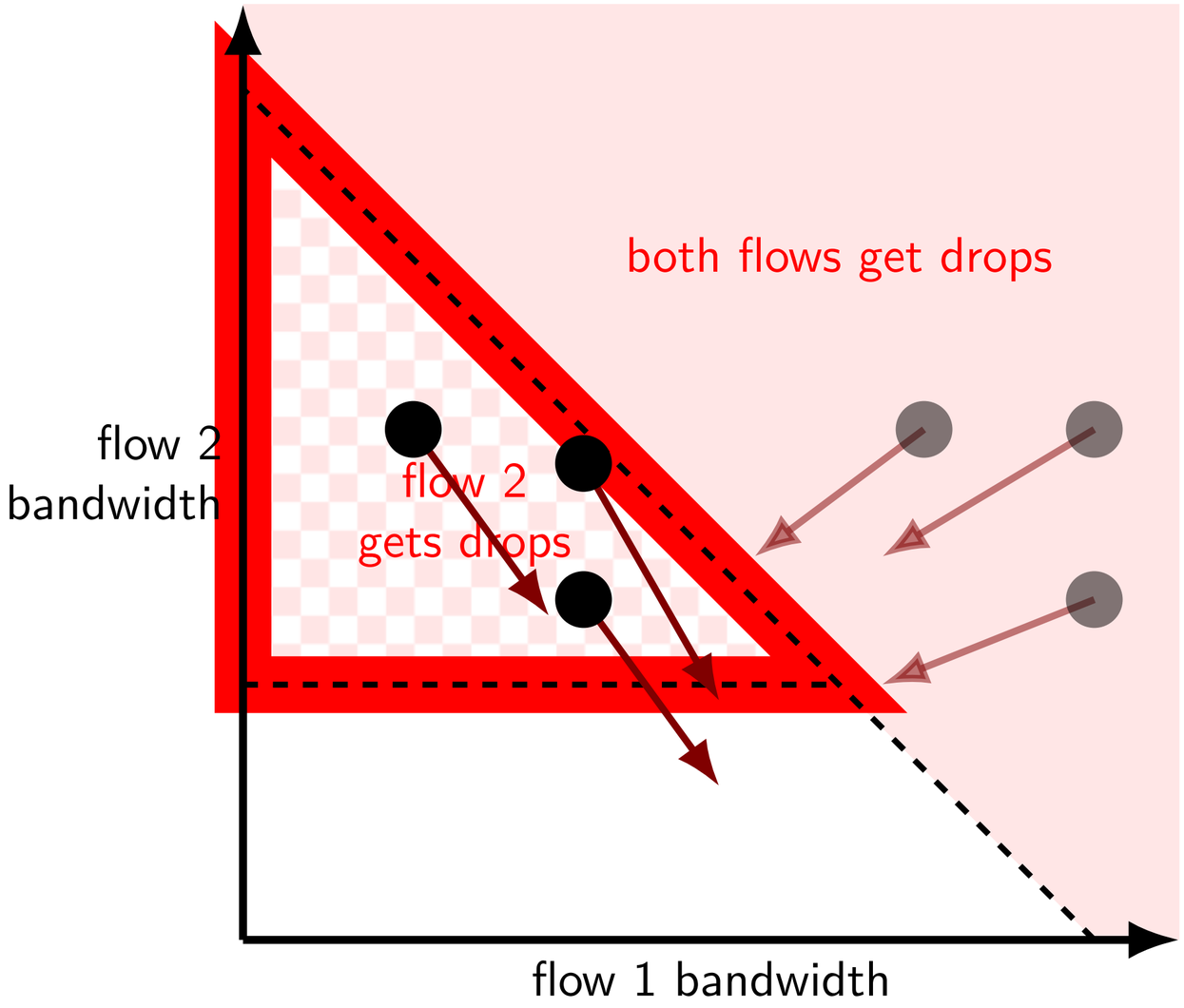


# slow link + mixed traffic



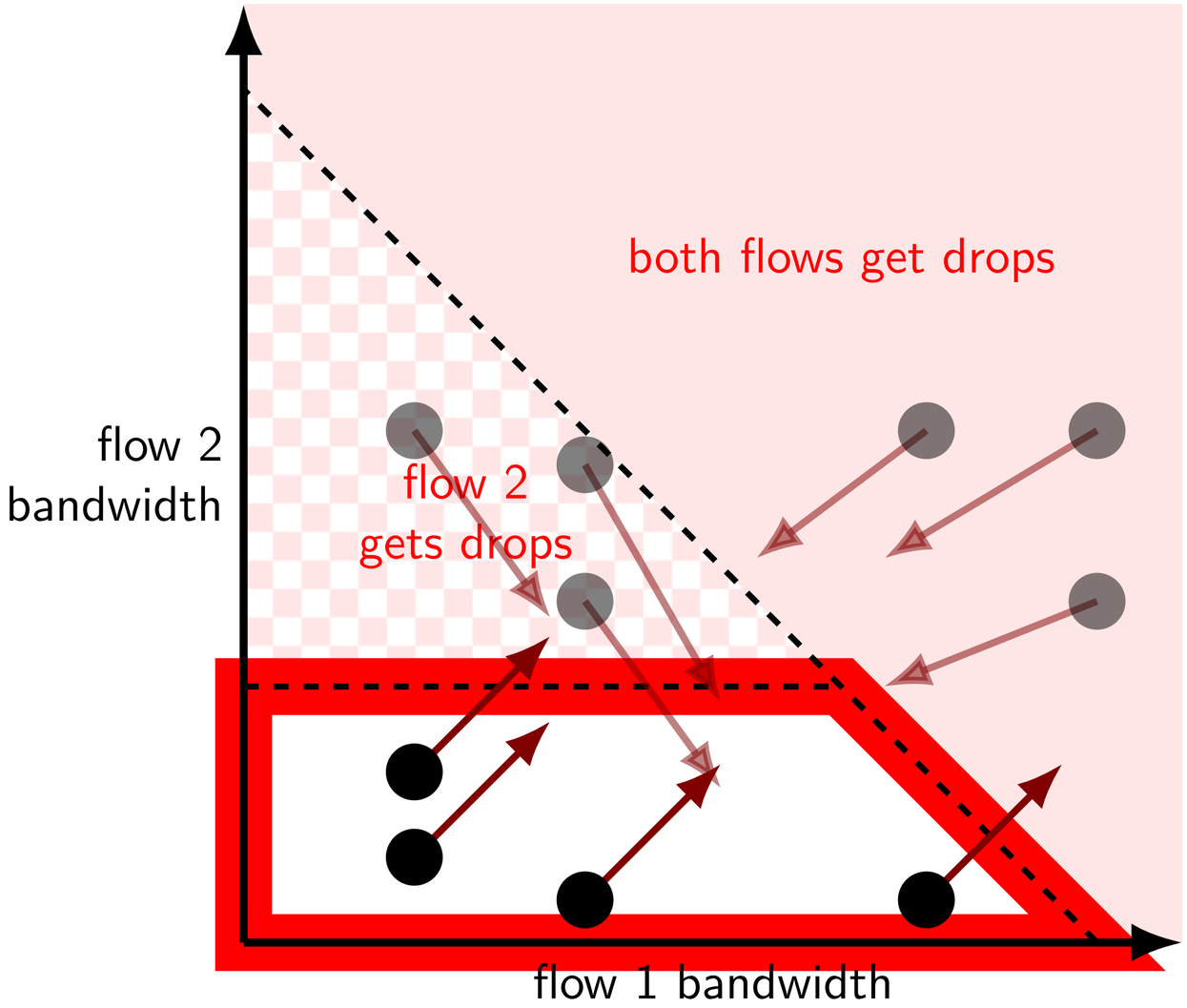

if both flows see drops,
multiplicative decrease
(toward origin)

# slow link + mixed traffic



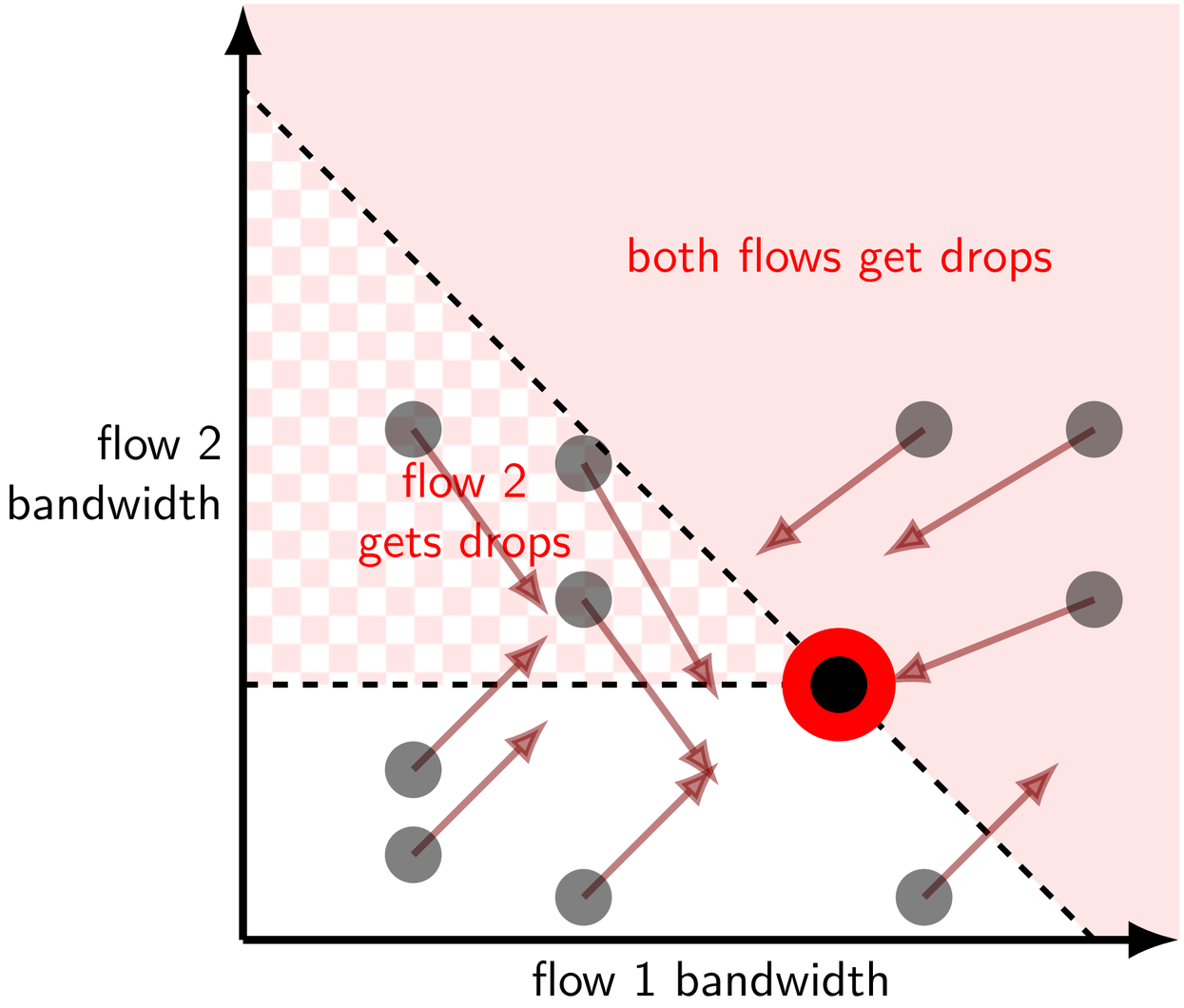

if only flow 2 see drops,
it decreases bandwidth and
flow 1 increase bandwidth

# slow link + mixed traffic




if both flows see no drops,
both increase additively
(45 degree angle)

# slow link + mixed traffic




result: flow 2 reaches
limit of slow link
flow 1 gets the rest
of the bandwidth

# fairness metrics

would like to say both allocations are ‘fair’

easy when ideal allocation is equal, but that’s not always the case

perhaps not equal, but most equal we can give on network

would like some way of formalizing this

# fairness intuition

unfair allocation = someone gets much less than others

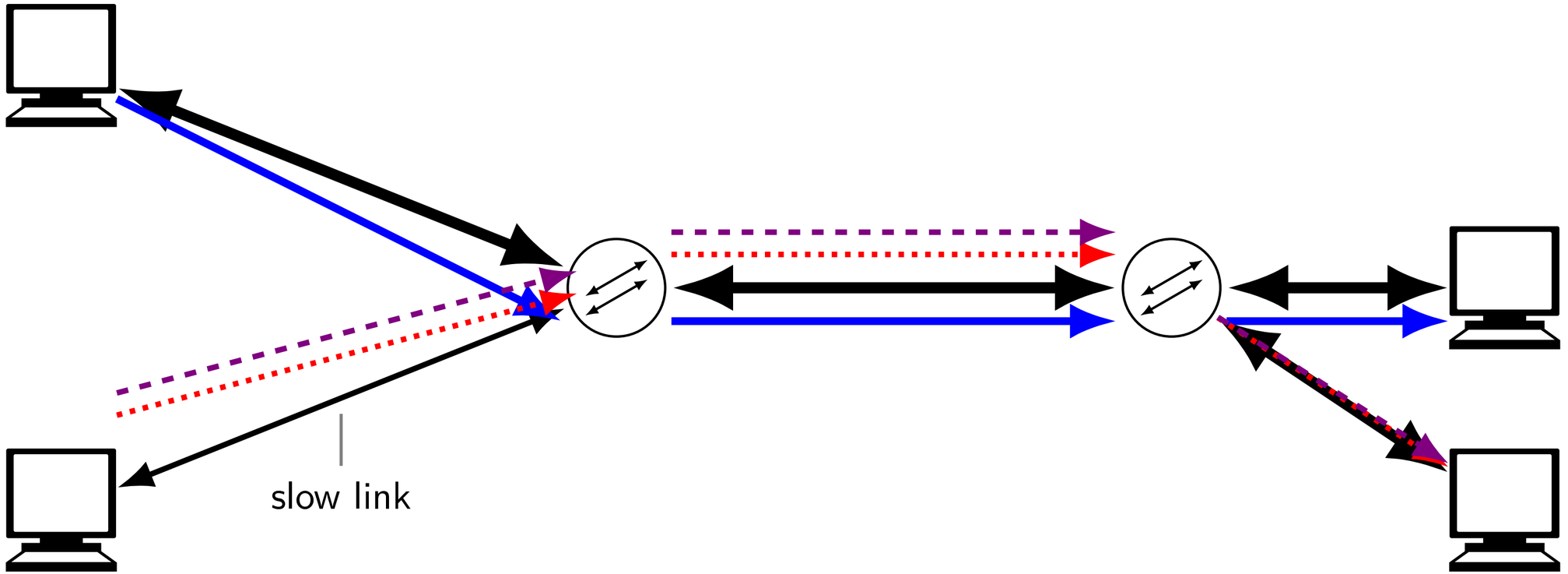
consequence: let's look for "starved" flow

if we can add to one flow…
and only hurt flows that are faster than it…
then that's "unfair"

that flow "deserves" more (possibly at expense of "richer" flows)

idea called *max-min fairness*

# exercise




let’s say slow link has 5 MByte/s capacity, other links 20MByte/s

why are these fair/unfair? (by min-max fairness)

solid = 10MByte, dotted = 2MByte/s, dashed = 3MByte/s

solid = 16MByte, dashed = 2MByte/s, dashed = 2MByte/s

# Jain's fairness index

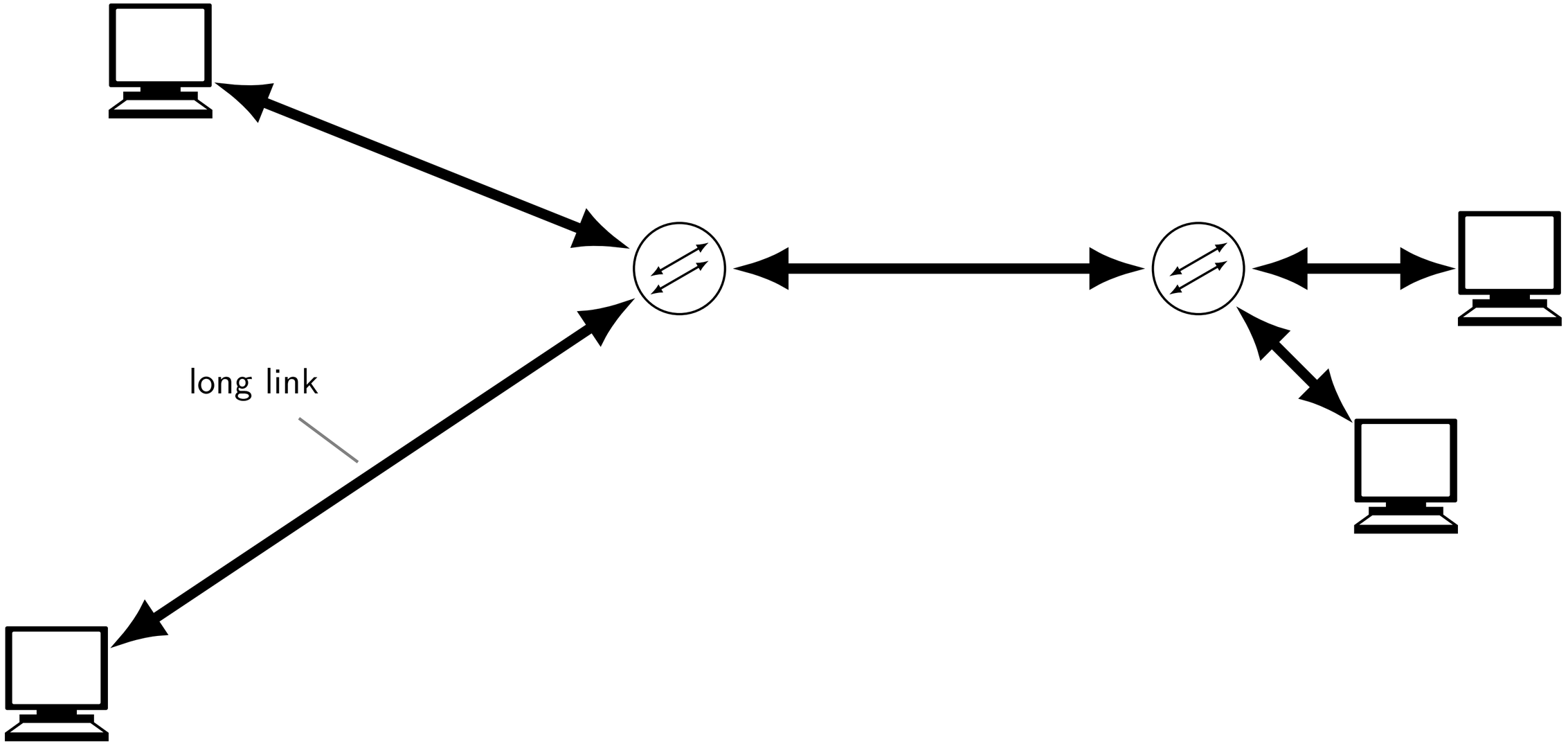
more common metric, but for scenarios where equal allocation makes sense

if $x_i$ is i'th flow's share:

$$\frac{(\sum x_i)^2}{n \cdot \sum x_i^2}$$

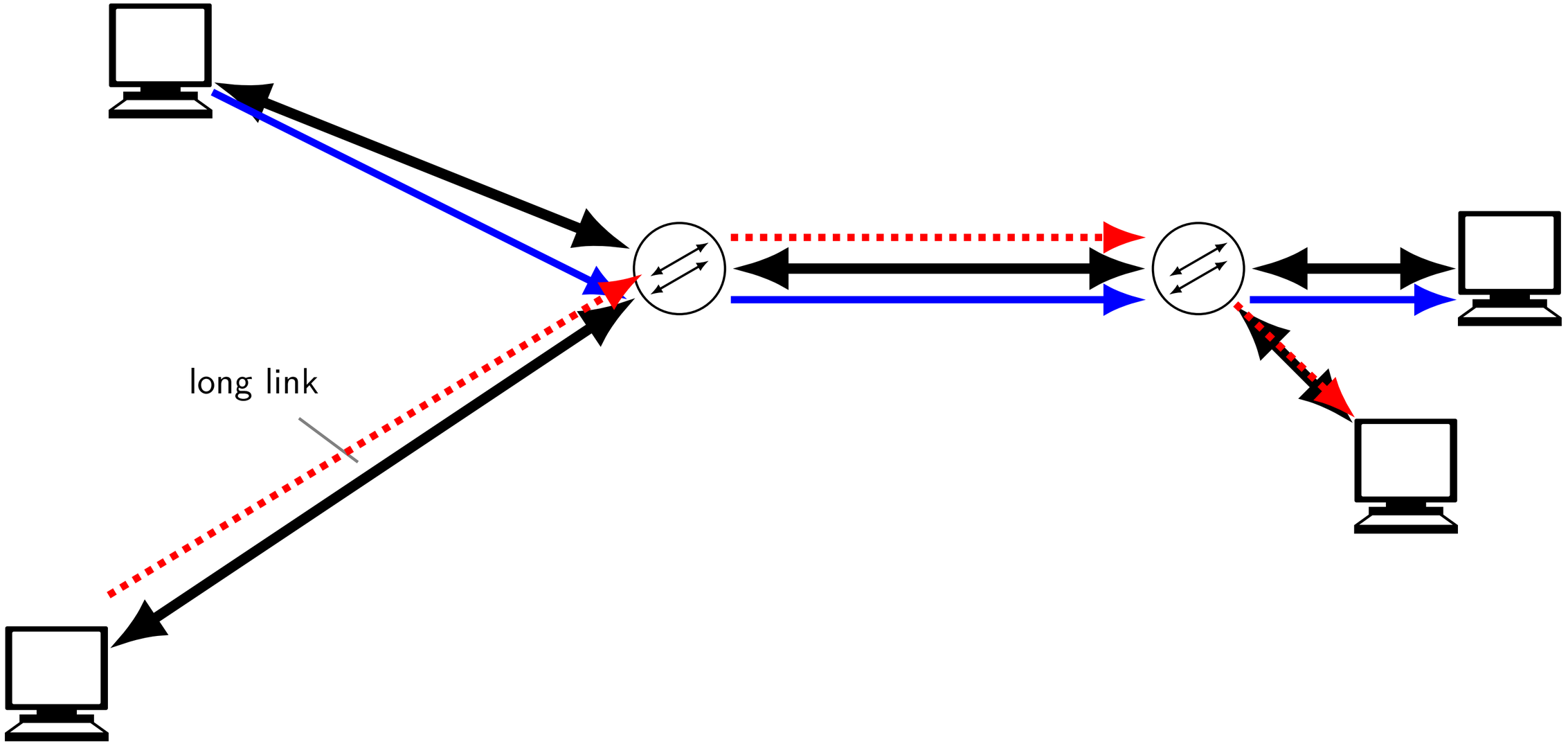
approaches 1 when allocations equal, $\frac{1}{n}$ if one flow gets everything

# long links



# long links

# long links



# exercise: window size?

flow 1: 500 packets/sec, 50 ms round trip

flow 2: 500 packets/sec, 100 ms round trip

exercise: what window size achieves this for each flow?

# exercise: window size?

flow 1: 500 packets/sec, 50 ms round trip

flow 2: 500 packets/sec, 100 ms round trip

exercise: what window size achieves this for each flow?

1: window size 25; 2: window size 50

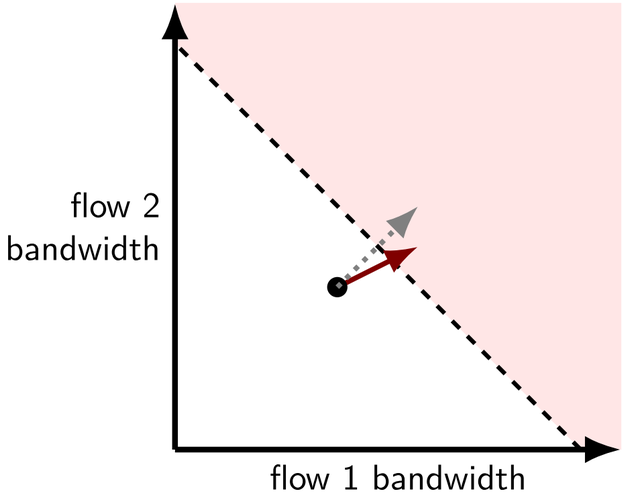
# revisiting additive increase

TCP: add +1 to window size each round trip time

flow 1: window 25+1 $\rightarrow$ 26 pkt/50 ms = 520 pkt/sec

flow 2: window 50+1 $\rightarrow$ 51 pkt/100 ms = 510 pkt/sec

using rough model from before:




flow 1 increases faster
than flow 2 increases
not 45-degree angle anymore

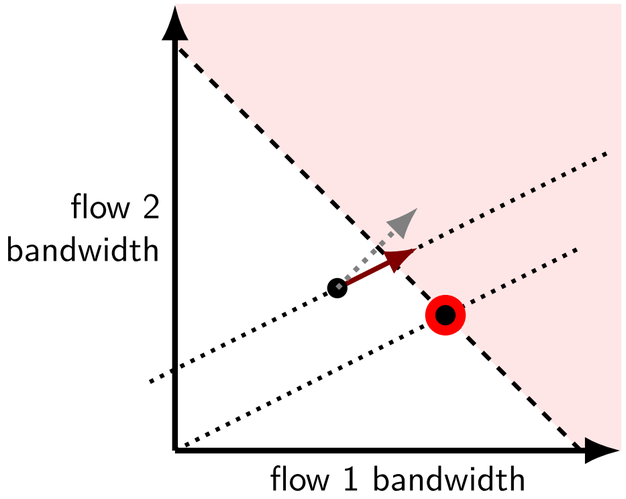
# revisiting additive increase

TCP: add +1 to window size each round trip time

flow 1: window 25+1 $\rightarrow$ 26 pkt/50 ms = 520 pkt/sec

flow 2: window 50+1 $\rightarrow$ 51 pkt/100 ms = 510 pkt/sec

using rough model from before:




in equilibrium
flow 1 gets more bandwidth

# other unfairness

lower round-trip gets more bandwidth

can also get more bandwidth by…

using more connections (‘independent’ windows)

adding more than + 1 packet to window size/RTT

# alternate congestion control

usually *more aggressive* than "normal" TCP

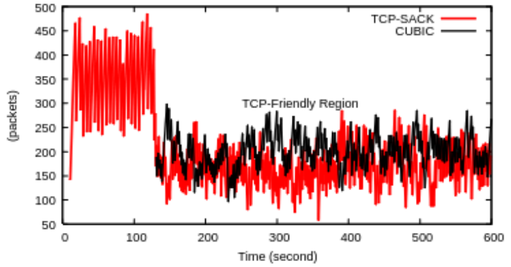
- examples we'll talk about later: CUBIC, Compound TCP
- (common defaults on recentish OSes)
- needed to fully utilize bandwidth on fast network

concern: are these "stealing" all the bandwidth from normal TCP?

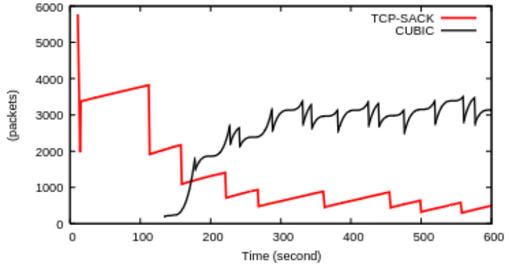
"TCP-friendly"

- should not make TCP used alongside them behave poorly

# examples: TCP-unfriendliness




(a) RTT 8ms.




(b) RTT 82ms.

**Figure 5: One CUBIC flow and one TCP-SACK flow. Bandwidth is set to 400Mbps.**

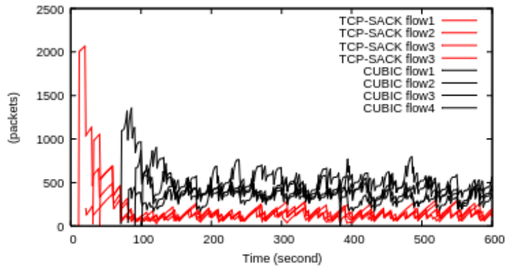



**Figure 6: Four TCP-SACK flows and four CUBIC flows over 40ms RTT**

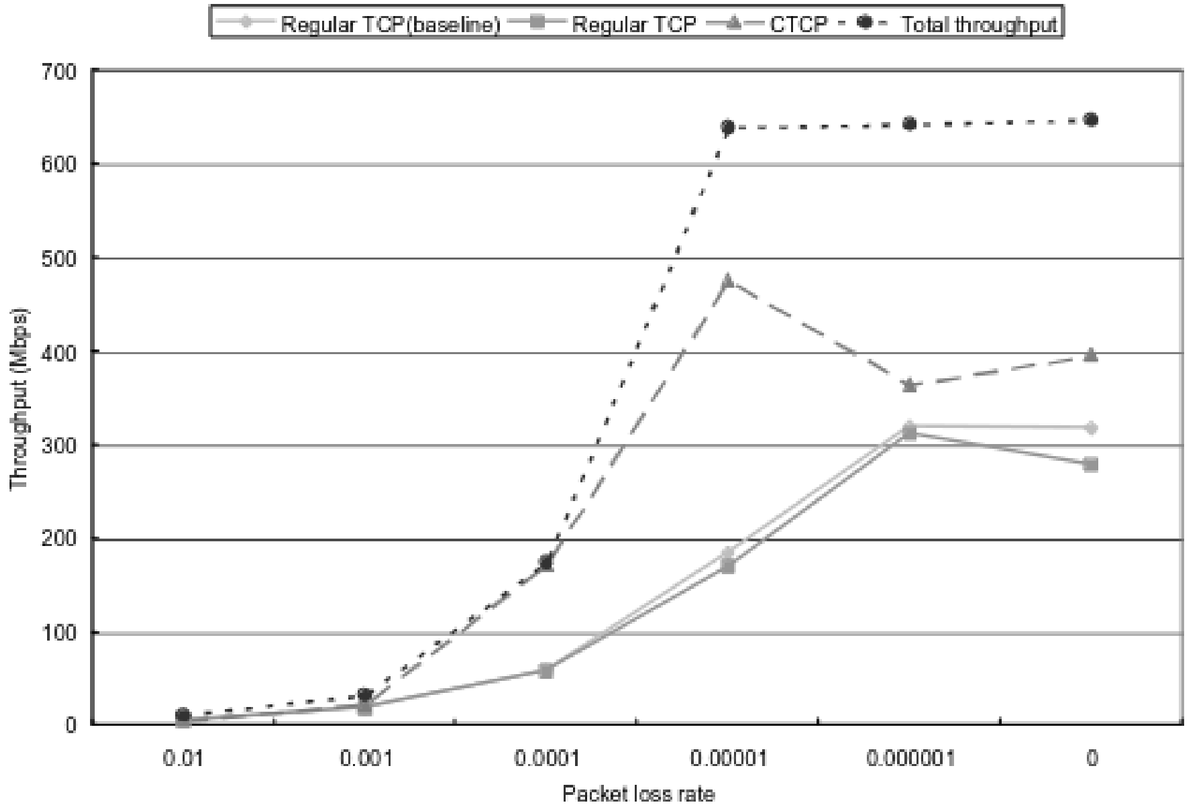



Figure 11. Throughput of CTCP and Regular TCP flows when competing for same bottleneck.

from Ha et al, “CUBIC: A New TCP-Friendly High-Speed TCP Variant”

from Tan et al, “A Compound TCP Approach for High-speed and Long Distance Networks”

# a theoretical result

for RTT-unfairness, standard TCP with selective acknowledgments:

(from Mathis et al, “The Macroscopeic Behavior of the TCP Congestion Avoidance Algorithm” (1997))

$$\text{throughput} \approx \text{constant} \times \frac{\text{packet size}}{\text{RTT}\sqrt{\text{loss rate}}}$$

# empirical results

some results from Philip (IMC'21)*

with same congestion control algorithm + RTT, Jain's fairness index > 0.99

CUBIC takes 70-80% of throughput when competing with equal number of traditional TCP flows

- cubic notably uses non-additive increase

*Philip, Ware, Athapathu, Sherry, Sekar, "Revisting TCP Congestion Control Throughput Models & Fairness Properties At Scale" (IMC'21)